Assessment Task 2: Data Mining Project Report

PCA-Informed K-Means Clustering of U.S. State Air Pollution (2000–2016): Robustness Across Python, WEKA, and NumPy Implementation

Chanutda Manosorn (s5313405)

3804ICT - (Applied) Data Mining Trimester 2

Griffith University, Gold Coast

Introduction

Air pollution remains one of the most persistent environmental challenges of the 21st century, with serious implications for human health, economic productivity, and climate action. International frameworks such as the Paris Agreement (2015) and the United Nations Sustainable Development Goal 13 (Climate Action) highlight the need for both global and local strategies to mitigate air pollutants and greenhouse gas emissions (United Nations, 2023). While global and national dashboards provide valuable summaries of air-quality trends, they often mask significant regional and seasonal differences within countries (Ritchie & Roser, 2024).

The United States provides a particularly relevant case for examining air pollution because of its diverse geography, extensive monitoring network, and evolving environmental policies, including amendments to the Clean Air Act and the years leading up to the Paris Agreement (Intergovernmental Panel on Climate Change [IPCC], 2023; International Federation of Red Cross and Red Crescent Societies [IFRC], 2024). The U.S. Pollution Dataset (Sogun, 2021), which contains over 1.7 million observations from 2000 to 2016, offers a unique opportunity to explore these patterns systematically. The dataset records four key pollutants including, NO_2, O_3, SO_2, and CO across hundreds of monitoring stations, along with metadata such as location, date, and air-quality index (AQI).

Research Questions include (1) *Do U.S. states naturally cluster into a small number of groups based on multi-year pollutant profiles (NO_2, O_3, SO_2, CO, and AQI)?* (2) *How robust are these clusters across analytical tools (Python, WEKA) and a hard-coded implementation?* The study aims to **cluster U.S. states into groups with similar pollutant conditions**, providing pattern-based evidence to support future policy and regional coordination. In interpreting the clusters, geographical context and shared climatic regimes are considered.

This study applies PCA with K-means to group U.S. states by pollutant profiles. The results provide interpretable, data-driven groupings (with k=2 selected by Silhouette) and a compact summary of pollution drivers via the first 4 principal components (more than 95% variance explained). While this study is not a policy evaluation, the analysis can inform future policy development and highlights shared-climate effects, neighbouring states often exhibit similar overall AQI patterns (see Figure 1).

Methods

Preliminary Data

Data Preparation

The preliminary analysis (Appendix A) established the overall quality and statistical characteristics of the U.S. air pollution dataset (2000–2016). Several important observations emerged:

- **Data quality:** Missing values were concentrated in CO AQI and SO_2 AQI (together approximately 75% of incomplete records), while duplicate records were absent. Negative pollutant values were identified in SO_2, CO, and NO_2 attributes, and corrected to NaN to ensure valid analyses.
- **Distributional patterns:** Histograms confirmed that NO_2, SO_2, and CO exhibit highly skewed distributions with extreme outliers, whereas O_3 displays a more symmetric, normal bell-shaped distribution.

- **Correlation structure:** Strong correlations exist between pollutant means and their AQI measures, with NO_2 and CO showing moderate to high co-variation, while O_3 behaves more independently due to its secondary photochemical nature.
- **Spatial variation:** Boxplots across the 10 most polluted states revealed that O_3 and NO_2 are dominant contributors to high AQI.
- **Temporal trends:** Median annual AQI values for NO_2, SO_2, and CO showed clear downward trajectories, consistent with emission-control policies, while O_3 remained relatively stable over time.

Together, these insights motivate PCA for dimensionality reduction and K-means state clustering to detect similarities in pollution dynamics across regions and years.

Clustering and Dimensionality Reduction (unsupervised)

Data Preparation

Preliminary data exploration

Preliminary analysis of the U.S. Pollution Dataset in Appendix A included data inspection, cleaning, and descriptive statistics across pollutants and AQI measures. After removing duplicates and correcting negative values to NaN, the cleaned dataset retained 428,062 valid records aggregated into 473 state-year profiles. Distribution plots and correlation checks confirmed that NO_2, SO_2, and CO were highly right skewed with many extreme values, whereas O_3 displayed a more symmetric bell-shaped distribution employed matplotlib and seaborn. Time series showed clear downward trajectories for NO_2, SO_2, and CO means (e.g., NO_2 declined from approximately 20 ppb in 2000 to <10 ppb by 2016), while O_3 remained relatively stable, supporting the validity of subsequent pattern analysis. The last file is saved into state_year_profiles.csv and state_year_X_scaled.npy for future implementation.

Evaluation Metrics

Clustering quality was assessed using unsupervised evaluation metrics to compare between 3 pipelines including Python library, WEKA and Hard code algorithm:

- Silhouette Score: Measures cohesion (intra-cluster similarity) and separation (inter-cluster distinctiveness). Scores range from −1 to 1, with higher values indicating more coherent clustering (GeeksforGeeks, 2025).
- Inertia (for K-Means): Captures the within-cluster sum of squares, providing insight into variance explained by cluster assignments and supporting the elbow method.

Implementation A — Python (libraries)

Detection of outliers and abnormal patterns

Outlier and pattern detection in Appendix B focused on identifying extreme pollutant values using CBLOF and visual diagnostics. Contamination rates of 10%, 5%, and 1% were tested, and the 5% setting was judged most credible and reasonable because scatterplots showed that higher thresholds incorrectly flagged points close to the central mean as anomalies. At 5%, outliers were confined to the far tails of the distributions, which is consistent with expected environmental extremes. The stability of these results across years confirmed that the cleaned dataset was both reliable and suitable for robust clustering and prediction.

PCA with K-Means clustering

PCA and K-Means clustering in Appendix C were applied to aggregated state-year profiles. The first 4 principal components together explained over 95% of the variance, using scikit-learn to check and ensuring minimal information loss. PC1 was dominated by NO_2 Mean (0.41), CO Mean (0.37), and NO_2 AQI (0.40), capturing general pollution intensity, while PC2 was strongly associated with O_3 Mean (0.70), distinguishing photochemical behaviour. K-Means clustering on these PCs (Silhouette score = 0.35465) produced 2 clearly separated state groups: Cluster 0 (high-pollution) with an average Overall AQI of around 41, and Cluster 1 (low-pollution) with an AQI of around 35. Most states consistently remained in the same cluster across years, confirming the robustness of the grouping and enabling meaningful interpretation in relation to state-level policy contexts.

The PCA and clustering analysis produced a clear two-group structure among U.S. states (Appendix C, Figure C2). Based on Silhouette scores (approximately 0.36) and the elbow method from inertia that Figure C1 shows 4 components covering over 95%, k=2 was identified as the most interpretable solution (Appendix C, Table C2). This partition provided the best balance between compactness within clusters and separation across them.

Implementation B — WEKA (Explorer)

Preprocess

Using the aggregated file state_year_profiles.csv in preprocessing and standardize all numeric attributes while filter other columns out e.g., Overall AQI to calculate PrincipalComponents with varianceCovered = 0.95 to retain 95% explained variance similar to Implementation A - Python transformed attributes No. 1–4 (see Appendix D, Figure D1).

Clustering

AddCluster and choose SimpleKMeans with numClusters = 2, Euclidean distance, seed = 42 (same as Python), default iterations. The output shows Clustered instances = 124/349 and Silhouette (WEKA labels) = 0.331 according to label switching property of K-means, but after flipping (0 and 1) agreement with sklearn rose to 89.6%.

Implementation C — Hard-code (NumPy, no Python library)

This section implements K-means and evaluation from first principles to verify correctness and provide transparent metrics (Sum Square Error - SSE and Silhouette) from mathematics formula to calculate.

K-Means clustering

The K-Means algorithm is coded in NumPy on state_year_X_scaled.npy from Data Preparation without PCA. Firstly, initialising with a random selection of k data points as centroids (seed = 42) and iterates assigned by Euclidean distance to recompute centroids and points distances (see Appendix E4). The formulas using are:

$$SSE = \sum_{i=1}^{n}(x_i - \bar{x})^2$$

$$Silhouette = \frac{b - a}{\max{(a, b)}}$$

The result shows SSE = 2787.68, Silhouette = 0.353 and Agreement = 99.4% compared to sklearn labels (Python), and 89.9% compared to WEKA (after aligning label IDs).

Implementation Results

PCA and K-means revealed a clear two-cluster structure in state-year pollution profiles. Across platforms, the solutions were consistent:

- **Python (libraries):** Silhouette 0.353; cluster sizes 143/330.
- **WEKA (Explorer):** Silhouette 0.331; cluster sizes 124/349. After label alignment (0 & 1), agreement with Python was about **89.6%**, confirming a closely matching partition.
- **Hard code (using NumPy):** Silhouette 0.353**;** SSE 2787.68; agreement with Python about 99.4% and with WEKA about 89.9%, validating the custom implementation.

Table 1. Python, WEKA, and Hard code results comparison

Method	k	Silhouette	Cluster sizes
Python (libraries)	2	0.353	143/330
WEKA (Explorer)	2	0.331	124/349
Hard code (with NumPy)	2	0.353	146/327

Table 1 shows that WEKA reproduces 2 coherent clusters and closely mirrors the Python outcome, supporting the robustness of the analysis across tools. The hard-coded implementation matches sklearn almost exactly (better than WEKA), confirming algorithmic correctness. Moreover, recovering the same 2 clusters on the non-PCA standardized indicates that the structure is stable even without dimensionality reduction; however, PCA mainly improves interpretability and reduces noise.

Cluster profiles

Given the high cross-method consistency (Python, WEKA, and the hard-coded implementation), the study adopts the Python solution as the canonical reference for downstream analyses, cluster profiles and state assignment, while retaining WEKA and hard-code results as robustness checks.

Cluster profiles in Table C3 (Appendix C) summarize the pollutant characteristics that differentiate the 2 groups. Cluster 0, which includes small group of states, is marked by higher mean concentrations of NO_2, SO_2, and CO, aligning with an Overall AQI of around 41. This group reflects relatively clean air conditions across much of the U.S. In contrast, Cluster 1, comprising many of all states, shows consistently elevated pollutant concentrations, particularly lower NO_2 and SO_2 values, resulting in an average Overall AQI of about 35. These differences confirm that the clustering approach effectively separates states into distinct pollution regimes in Figure C2 (Appendix C), with Cluster 0 reflecting more industrialized or urbanized air quality profiles, and Cluster 1 capturing cleaner environmental profiles.

State assignment

State assignment results in Table C4 demonstrate how each U.S. state was classified into its dominant cluster across years. Most states (39 out of 45) consistently fell into Cluster 1, highlighting stability in their lower-pollution profiles. The remaining 6 states, including Arizona, Colorado, Michigan, New Jersey, New York and Missouri were persistently assigned to Cluster 0, reflecting elevated pollutant levels and higher AQI values. While Table C4 (Appendix C) confirms cluster assignments, Table C5 (Appendix C) provides detailed pollutant concentrations for each state, ranked by Overall AQI, which aligns with the observed separation between Cluster 0 and Cluster 1. The consistency of these assignments across multiple years confirms the robustness of the clustering solution, with minimal fluctuation between groups, thereby reinforcing the validity of the two-cluster structure.

Results for the Research Questions

(1) Do U.S. states naturally cluster into a small number of groups based on multi-year pollutant profiles (NO_2, O_3, SO_2, CO, and AQI)?
Yes. The best solution is k = 2 on state–year profiles. In Python: Silhouette = 0.353, cluster sizes 143/330. Profiles differ clearly:

- **Cluster 0 (small group):** higher NO_2/SO_2/CO means, higher Overall AQI (approx. 41).
- **Cluster 1 (large group):** lower pollutant means, lower Overall AQI (approx. 35). PCA shows drivers: PC1 (general pollution intensity: NO_2 Mean, NO_2 AQI, CO Mean and SO_2 AQI) and PC2 (O_3 Mean and O_3 AQI).

(2) How robust are these clusters across analytical tools (Python, WEKA) and a hard-coded implementation?
Yes. The two-cluster structure is consistent across toolkits and algorithm implementations (Table 1):

- **Python (library):** Silhouette 0.353.
- **WEKA:** Silhouette 0.331; after label flip has 89.6% agreement with Python.
- **Hard code (NumPy):** Silhouette 0.353, SSE 2787.68 with 99.4% agreement with Python (and 89.9% similarity with WEKA).

Finding confirm the same structure between using the non-PCA standardized space (hard code) and PCA mainly encourages the pattern interpretability and noise reduction.

Discussion

This discussion compares 2 complementary visualizations: a clustering map of U.S. states derived from PCA-based k-means with k=2 (Silhouette score approximately 0.36) and a choropleth of mean Overall AQI by state. The choice of k=2 follows the highest silhouette score among k [2, 10] in Table C2 (Appendix C), and 4 principal components (PCs) together explain over 95% of the total variance (Appendix C, Figure C1), ensuring that the reduced feature space retains the dominant signal.

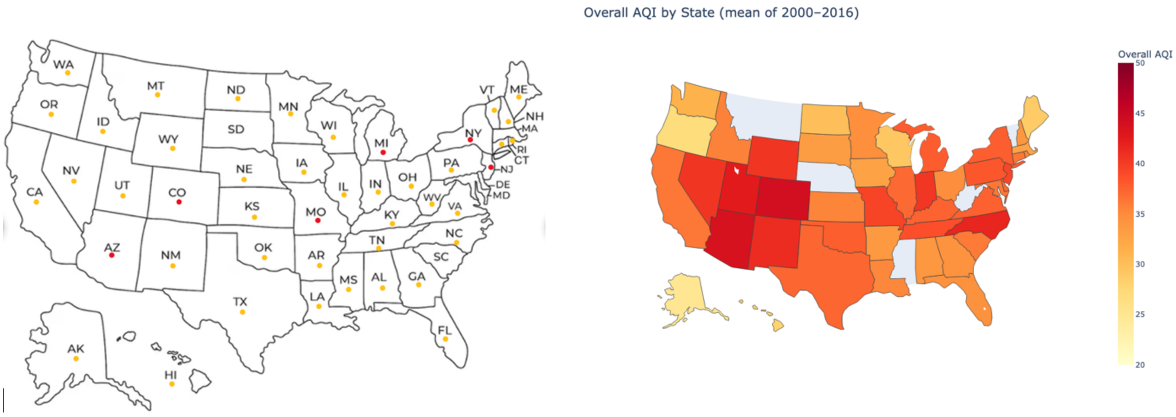

Figure 1. Average Overall AQI and Cluster Assignments of U.S. States (2000–2016)

Figure 1 (left) depicts state assignments (k=2) by outlined colour-dots, with red denoting Cluster 0 (higher-pollution profile) and yellow denoting Cluster 1 (lower-pollution profile). A smaller set of states in cluster 0, such as Arizona, Colorado, Michigan, Missouri, New Jersey, and New York, persistently fall into the higher-pollution cluster, while most states group into the lower-pollution cluster (Table 2).

The 2 maps need not match one-for-one because they summarise different constructs. The AQI choropleth (Figure 1, right) is univariate, Overall AQI aggregates pollutant-specific indices at the observation level and is then averaged at the state level, whereas the clustering is multivariate, formed from standardised pollutant means and AQIs projected into PC perspective. Consequently, a state with a relatively high Overall AQI can still be grouped with “lower” states if its multi-pollutant profile (as weighted by the PCs) resembles those cleaner states, e.g., 'North Carolina' – NC is in cluster 1 (yellow), while its overall AQI is in a red tone. In other words, cluster membership reflects patterns across pollutants, not the magnitude of a single composite index.

PC loadings clarify the mechanism. Table 2 shows that PC1 places substantial weight on NO_2 and CO measures (e.g., NO_2 Mean/AQI, CO AQI), PC2 is dominated by O_3 (O_3 Mean/AQI), and PC3–PC4 capture residual SO_2/CO structure. These axes, which differentially weight pollutant families, govern distances in the reduced feature space that k-means partitions. States can therefore separate because they differ on which pollutant family drives their profile, even when Overall AQI levels are comparable. This explains why clustering yields a more nuanced grouping that does not always mirror the AQI map directly.

Geographic context further explains the visuals. Figure D1 (Appendix D) exhibits spatial contiguity – adjacent states often share similar colours – because air masses, regional emission patterns, and topography create cross-border spillovers and all regions “shared air” together. From a policy standpoint, the cluster map identifies what multi-pollutant regimes exist, while the AQI map indicates where intensity concentrates. Taken together, the evidence supports regional, multi-state coordination to address transboundary drivers alongside state-specific measures tailored to each cluster's dominant pollutants. The clustering analysis thus provides a structural lens on pollutant relationships, and the AQI map situates those structures in their geographic context; both perspectives are necessary for policy-relevant interpretation.

Table 2. Main principal components in 4 PC for K-Means clustering

Component	PC1	PC2	PC3	PC4
NO_2 Mean	**0.417210**	-0.028603	-0.066480	-0.457316
SO_2 Mean	0.351842	0.021547	**0.600156**	0.248053
CO Mean	0.355825	0.027488	-0.421558	**0.528553**
O_3 Mean	-0.241362	**0.528513**	0.089507	0.139861
NO_2 AQI	**0.406780**	0.047167	-0.115051	-0.491511
SO_2 AQI	**0.375143**	0.097344	**0.526459**	0.168375
CO AQI	**0.401037**	0.043779	-0.387663	**0.331091**
O_3 AQI	-0.101493	**0.624245**	-0.065530	0.018615

There are studies, particularly those in geography and climate (Waller, 2021; Randalls, 2017), showing that regions effectively share a climate that transcends administrative borders. Waller (2021) points out that geographical and meteorological factors are the primary drivers shaping spatial gradients in pollutant concentrations. In practice, the movement of air masses and prevailing regional circulation patterns create a shared environmental field that cannot be contained by city or state lines; this is why adjacent states in Figure 1 (right) display similar or closely aligned Overall AQI values. Randalls (2017) likewise underscores that policy design must account for this shared-climate influence, since both air pollution and climate effects routinely flow across traditional jurisdictions. Acknowledging that neighbouring regions share a climatic context enables more cohesive, cooperative policies that reflect these interconnected environmental realities.

Finally, Feng et al. (2019) emphasises that climatic factors, such as air-mass regimes, temperature, and humidity, govern the formation, dispersion, and persistence of pollutants, supporting long-horizon, state-level policy analysis. In our application, the relative weights of the first 4 principal components (PCs) representing over 95% of AQI for each pollutant provide a compact, interpretable summary of these drivers, and can be used to compare states systematically.

Acknowledgements

This report partially applied coding assistance by ChatGPT (OpenAI) to scaffold K-Means and Silhouette implementations for construction purposes. All ideas, analysis, verification, and final conclusions and discussion are the author's work. Use of AI followed the academic integrity policy.

References

Feng, H., Zou, B., Wang, J., & Gu, X. (2019). Dominant variables of global air pollution-climate interaction: Geographic insight. *Ecological indicators*, *99*, 251-260.

GeeksforGeeks. (2025, June 23). *What is Silhouette Score?* GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/what-is-silhouette-score/

Igobwa, A. M., Gachanja, J., Muriithi, B., Olukuru, J., Wairegi, A., & Rutenberg, I. (2022). A canary, a coal mine, and imperfect data: determining the efficacy of open-source climate change models in detecting and predicting extreme weather events in Northern and Western Kenya. *Climatic Change*, *174*(3), 24.

Intergovernmental Panel on Climate Change. (2023). *AR6 synthesis report: Climate change 2023*. https://www.ipcc.ch/report/ar6/syr/

International Federation of Red Cross and Red Crescent Societies. (2024). *Global Climate Resilience Platform: Status update report 2024*. https://www.ifrc.org/sites/default/files/2024-03/GCaRP-Status-Update-2024.pdf

Randalls, S. (2017). Contributions and perspectives from geography to the study of climate. *Wiley Interdisciplinary Reviews: Climate Change*, *8*(4), e466.

Ritchie, H., & Roser, M. (2024). *Climate belief and policy support* [Data set]. Our World in Data, University of Oxford. https://ourworldindata.org/

scikit-learn. (n.d.). *silhouette_score*. Retrieved September 18, 2025, from https://scikit learn.org/stable/modules/generated/ sklearn.metrics.silhouette_score.html

Sogun, S. (2021). *U.S. Pollution Dataset (2000–2016)* [Data set]. Kaggle. https://www.kaggle.com/datasets/sogun3/uspollution

Stanford HLab. (n.d.). *Error sum of squares (SSE).* Retrieved September 18, 2025, from https://hlab.stanford.edu/brian/error_ sum_of_squares.html

United Nations. (2023). *The sustainable development goals report 2023*. https://unstats.un.org/sdgs/report/2023/

U.S. Environmental Protection Agency. (2024). *Technical assistance document for the reporting of daily air quality – the Air Quality Index (AQI)* (EPA-454/B-24-002). Office of Air Quality Planning and Standards. https://document.airnow.gov/technical-assistance-document-for-the-reporting-of-daily-air-quailty.pdf

Waller, R. (2021). Geography matters.

University of Waikato. (n.d.). *The Weka Workbench.* Retrieved September 18, 2025, from https://ml.cms.waikato.ac.nz/weka/

Appendices

Appendix A – Data Preparation

A1. Setup & Load

```
import pandas as pd
import numpy as np
# Load dataset
df = pd.read_csv("pollution_us_2000_2016.csv", low_memory=False)
# --- Filter only U.S. states (+DC) -> Original data has Mexico and Columbia
US_STATES = {
    "Alabama","Alaska","Arizona","Arkansas","California","Colorado","Connecticut","Delaware",
    "Florida","Georgia","Hawaii","Idaho","Illinois","Indiana","Iowa",
    "Kansas","Kentucky","Louisiana","Maine","Maryland","Massachusetts","Michigan","Minnesota",
    "Mississippi","Missouri","Montana","Nebraska","Nevada","New Hampshire","New Jersey",
    "New Mexico","New York","North Carolina","North Dakota","Ohio","Oklahoma","Oregon",
    "Pennsylvania","Rhode Island","South Carolina","South Dakota","Tennessee","Texas","Utah",
    "Vermont","Virginia","Washington","West Virginia","Wisconsin","Wyoming"}
df["State"] = df["State"].astype(str).str.strip()
df = df[df["State"].isin(US_STATES)].copy()
print("Shape after filtering:", df.shape)
print("Unique states:", sorted(df["State"].unique())[:10], "...")
df.head()

df.to_csv("pollution_us_states_cleaned.csv", index=False)
```

The dataset *pollution_us_2000_2016.csv* was imported using pandas for preprocessing and analysis. An initial inspection confirmed that the dataset contains 1,711,459 rows and 29 attributes, consistent with the description provided in the project proposal. These attributes include station identifiers (State Code, County Code, Site Number), geographical information (Address, State, County, City), temporal data (Date Local), pollutant measures (mean values, first maximum values, AQI), and unit descriptors for each pollutant. During inspection, records from Mexico and Colombia were found to be included in the raw file; these were removed to ensure the analysis focuses solely on U.S. states.

Table A1 shows the full list of column headers extracted from the raw dataset. This confirms that the dataset schema matches the Kaggle source (Sogun, 2021) and provides a reliable foundation for subsequent data cleaning and transformation steps.

Table A1. Column headers in the U.S. Pollution Dataset (25 samples)

		State Code	County Code	Site Num	Address	State	County	City	Date Local	NO2 Units	NO2 Mean	NO2 1st Max Value
1	0	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-01	Parts per billion	19.041667	49.0
2	1	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-01	Parts per billion	19.041667	49.0
3	2	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-01	Parts per billion	19.041667	49.0
4	3	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-01	Parts per billion	19.041667	49.0
5	4	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-02	Parts per billion	22.958333	36.0
6	5	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-02	Parts per billion	22.958333	36.0
7	6	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-02	Parts per billion	22.958333	36.0
8	7	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-02	Parts per billion	22.958333	36.0
9	8	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-03	Parts per billion	38.125	51.0
10	9	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-03	Parts per billion	38.125	51.0
11	10	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-03	Parts per billion	38.125	51.0
12	11	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-03	Parts per billion	38.125	51.0
13	12	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-04	Parts per billion	40.26087	74.0
14	13	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-04	Parts per billion	40.26087	74.0
15	14	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-04	Parts per billion	40.26087	74.0
16	15	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-04	Parts per billion	40.26087	74.0
17	16	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-05	Parts per billion	48.45	61.0
18	17	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-05	Parts per billion	48.45	61.0
19	18	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-05	Parts per billion	48.45	61.0
20	19	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-05	Parts per billion	48.45	61.0
21	20	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-06	Parts per billion	39.95	73.0
22	21	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-06	Parts per billion	39.95	73.0
23	22	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-06	Parts per billion	39.95	73.0
24	23	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-06	Parts per billion	39.95	73.0
25	24	4	13	3002	1645 E ROOSEVELT ST-CENTRAL PHOENIX STN	Arizona	Maricopa	Phoenix	2000-01-07	Parts per billion	29.625	43.0

Table A1. Column headers in the U.S. Pollution Dataset (continue)

	NO2 1st Max Hour	NO2 AQI	O3 Units	O3 Mean	O3 1st Max Value	O3 1st Max Hour	O3 AQI	SO2 Units	SO2 Mean	SO2 1st Max Value	SO2 1st Max Hour	SO2 AQI	CO Units
1	19	46	Parts per million	0.0225	0.04	10	34	Parts per billion	3.0	9.0	21	13.0	Parts per million
2	19	46	Parts per million	0.0225	0.04	10	34	Parts per billion	3.0	9.0	21	13.0	Parts per million
3	19	46	Parts per million	0.0225	0.04	10	34	Parts per billion	2.975	6.6	23		Parts per million
4	19	46	Parts per million	0.0225	0.04	10	34	Parts per billion	2.975	6.6	23		Parts per million
5	19	34	Parts per million	0.013375	0.032	10	27	Parts per billion	1.958333	3.0	22	4.0	Parts per million
6	19	34	Parts per million	0.013375	0.032	10	27	Parts per billion	1.958333	3.0	22	4.0	Parts per million
7	19	34	Parts per million	0.013375	0.032	10	27	Parts per billion	1.9375	2.6	23		Parts per million
8	19	34	Parts per million	0.013375	0.032	10	27	Parts per billion	1.9375	2.6	23		Parts per million
9	8	48	Parts per million	0.007958	0.016	9	14	Parts per billion	5.25	11.0	19	16.0	Parts per million
10	8	48	Parts per million	0.007958	0.016	9	14	Parts per billion	5.25	11.0	19	16.0	Parts per million
11	8	48	Parts per million	0.007958	0.016	9	14	Parts per billion	5.2	8.3	20		Parts per million
12	8	48	Parts per million	0.007958	0.016	9	14	Parts per billion	5.2	8.3	20		Parts per million
13	8	72	Parts per million	0.014167	0.033	9	28	Parts per billion	7.083333	16.0	8	23.0	Parts per million
14	8	72	Parts per million	0.014167	0.033	9	28	Parts per billion	7.083333	16.0	8	23.0	Parts per million
15	8	72	Parts per million	0.014167	0.033	9	28	Parts per billion	7.05	12.6	8		Parts per million
16	8	72	Parts per million	0.014167	0.033	9	28	Parts per billion	7.05	12.6	8		Parts per million
17	22	58	Parts per million	0.006667	0.012	9	10	Parts per billion	8.708333	15.0	7	21.0	Parts per million
18	22	58	Parts per million	0.006667	0.012	9	10	Parts per billion	8.708333	15.0	7	21.0	Parts per million
19	22	58	Parts per million	0.006667	0.012	9	10	Parts per billion	8.7	14.0	8		Parts per million
20	22	58	Parts per million	0.006667	0.012	9	10	Parts per billion	8.7	14.0	8		Parts per million
21	8	71	Parts per million	0.01175	0.025	10	21	Parts per billion	6.761905	17.0	7	24.0	Parts per million
22	8	71	Parts per million	0.01175	0.025	10	21	Parts per billion	6.761905	17.0	7	24.0	Parts per million
23	8	71	Parts per million	0.01175	0.025	10	21	Parts per billion	7.066667	14.6	8		Parts per million
24	8	71	Parts per million	0.01175	0.025	10	21	Parts per billion	7.066667	14.6	8		Parts per million
25	9	41	Parts per million	0.011625	0.024	10	20	Parts per billion	8.666667	21.0	7	30.0	Parts per million

Table A1. Column headers in the U.S. Pollution Dataset (continue)

CO 1st Max Value	CO 1st Max Hour	CO AQI
4.2	21	
2.2	23	25.0
4.2	21	
2.2	23	25.0
1.6	23	
2.3	0	26.0
1.6	23	
2.3	0	26.0
4.4	8	
2.5	8	28.0
4.4	8	
2.5	8	28.0
5.1	21	
3.0	23	34.0
5.1	21	
3.0	23	34.0
5.6	7	
3.7	2	42.0
5.6	7	
3.7	2	42.0
6.3	7	
3.6	9	41.0
6.3	7	
3.6	9	41.0
6.4	8	

A2. Schema check, missing values, and duplicates

```
attribute_types = df.dtypes.value_counts()
summary_table = pd.DataFrame({
    'attribute type': attribute_types.index.astype(str),
    'count': attribute_types.values,
    'columns': [', '.join(df.columns[df.dtypes == dtype]) for dtype in attribute_types.index]
})
print(summary_table)

missing = df.isna().sum().sort_values(ascending=False)
print(missing.head(15))

dup_count = df.duplicated().sum()
print("Duplicate rows:", dup_count)
```

The schema inspection confirmed that the dataset consists of 29 attributes with three primary data types: *int64* (10 columns), *float64* (10 columns), and *object* (9 columns). Categorical attributes (e.g., State, County, City, Date Local, pollutant units) are stored as objects, while pollutant concentration and AQI values are represented as floats or integers.

A missing value audit revealed that most missing data are mainly concentrated in 2 variables: **CO AQI (855,735 rows missing)** and **SO_2 AQI (855,320 rows missing)**. These 2 attributes together account for nearly 75% of rows with incomplete information, which is consistent with the preliminary data audit repeated in Task 1. All other attributes show complete or near-complete coverage as shown in Figure A2. Finally, a duplicate row check indicated that there were no exact duplicate records in the dataset.

```
  attribute type  count                                            columns
0          int64     10  Unnamed: 0, State Code, County Code, Site Num,...
1        float64     10  NO2 Mean, NO2 1st Max Value, O3 Mean, O3 1st M...
2         object      9  Address, State, County, City, Date Local, NO2 ...
CO AQI               868582
SO2 AQI              868155
O3 Mean                   0
CO 1st Max Hour           0
CO 1st Max Value          0
CO Mean                   0
CO Units                  0
SO2 1st Max Hour          0
SO2 1st Max Value         0
SO2 Mean                  0
SO2 Units                 0
O3 AQI                    0
O3 1st Max Hour           0
O3 1st Max Value          0
Unnamed: 0                0
dtype: int64
Duplicate rows: 0
```

Figure A2. Data schema and completeness check

A3. Identify and correct negative pollutant values

```
# include O3 columns as well
pollutant_columns = [
    "NO2 Mean", "NO2 1st Max Value",
    "SO2 Mean", "SO2 1st Max Value",
    "CO Mean",  "CO 1st Max Value",
    "O3 Mean",  "O3 1st Max Value"]
# keep only columns that actually exist
cols_present = [c for c in pollutant_columns if c in df.columns]
# count negative values
invalid_value_summary = (df[cols_present] < 0).sum().sort_values(ascending=False)
print(invalid_value_summary[invalid_value_summary > 0])
# convert negatives to NaN
df[cols_present] = df[cols_present].mask(df[cols_present] < 0)
```

An audit of pollutant attributes revealed the presence of negative values across several variables. These are physically implausible concentrations resulting from measurement or reporting errors. To ensure data quality, all negative entries were converted to NaN, thereby preventing bias in subsequent analyses. All other pollutant attributes contained valid non-negative values. Notably, both O_3 Mean and O_3 1st Max Value contained zero negative entries, indicating no implausible measurements for ozone.

The audit showed that the following attributes contained negative values:

- SO_2 Mean: 22,588 instances
- SO_2 1st Max Value: 8,278 instances
- CO Mean: 1,018 instances
- NO_2 Mean: 896 instances
- CO 1st Max Value: 196 instances
- NO_2 1st Max Value: 184 instances

A4. Five-number summary for pollutant attributes

```
poll_cols = [c for c in df.columns if any(x in c for x in ["NO2","O3","SO2","CO"])]
table_a4 = df[poll_cols].describe(percentiles=[.25,.5,.75]).T[["min","25%","50%","75%","max"]]
table_a4
```

The 5 numbers summary (minimum, first quartile, median, third quartile, and maximum) was calculated for the pollutant attributes and their corresponding AQI values. This summary provides a concise overview of the distribution and spread of the data. Key observations include:

- NO_2: Mean concentrations range from 0.0 to 139.5 ppb, with a median around 10.6 ppb. The AQI values show a median of 22, confirming that NO_2 frequently contributes to moderate air-quality levels.
- O_3: Distributions are closer to normal, with concentrations between 0 and 0.095 ppm, and AQI values ranging up to 218. The quartiles indicate a steady central distribution.

- SO_2: Concentrations are mostly very low (median approximately 1.0 ppb), but maximum values exceed 321.7 ppb, reflecting occasional extreme pollution episodes. AQI values are typically low (median = 3), highlighting that SO_2 rarely drives poor air quality in most regions.
- CO: Median mean concentration is around 0.29 ppm, with occasional peaks above 7.5 ppm. AQI values are modest (median = 5), though some locations report AQI above 201 during extreme events.

Overall, the table highlights right-skewed distributions for NO_2, SO_2, and CO, while O_3 demonstrates a more symmetric distribution. These findings are consistent with established atmospheric pollution patterns and reinforce the need for preprocessing and binning prior to pattern mining.

	min	25%	50%	75%	max
NO2 Mean	0.0	5.691304	10.608696	17.554167	139.541667
NO2 1st Max Value	0.0	13.000000	23.400000	35.000000	267.000000
NO2 1st Max Hour	0.0	5.000000	9.000000	20.000000	23.000000
NO2 AQI	0.0	12.000000	22.000000	33.000000	132.000000
O3 Mean	0.0	0.017958	0.025958	0.033958	0.095083
O3 1st Max Value	0.0	0.029000	0.038000	0.048000	0.140000
O3 1st Max Hour	0.0	9.000000	10.000000	11.000000	23.000000
O3 AQI	0.0	25.000000	33.000000	42.000000	218.000000
SO2 Mean	0.0	0.270833	0.995833	2.300000	321.625000
SO2 1st Max Value	0.0	0.900000	2.000000	5.000000	351.000000
SO2 1st Max Hour	0.0	4.000000	8.000000	14.000000	23.000000
SO2 AQI	0.0	1.000000	3.000000	9.000000	200.000000
CO Mean	0.0	0.183333	0.289583	0.458333	7.508333
CO 1st Max Value	0.0	0.280000	0.400000	0.700000	19.900000
CO 1st Max Hour	0.0	0.000000	6.000000	13.000000	23.000000
CO AQI	0.0	2.000000	5.000000	7.000000	201.000000

Table A4. Five-number summary statistics for pollutant means, maximum values, and AQI measures

A5. Missing data heatmap

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12,4))
sns.heatmap(df.isna(), cbar=False, yticklabels=False, cmap="viridis")
plt.title("Heatmap of Missing Data (Yellow = Missing)")
plt.tight_layout()
plt.show()
```

The missing data heatmap provides a visual overview of attribute completeness across the dataset. Each vertical stripe corresponds to one column, with purple indicating complete data and yellow highlighting missing values. The figure confirms that most variables are fully populated, while missingness is concentrated in CO AQI and SO_2 AQI, each with over 868,000 missing entries, and a small fraction in SO_2 Mean.

Together, these AQI-related gaps account for most incomplete records, while the remaining pollutant and metadata fields show near-complete coverage. This diagnostic confirms that missingness is highly localised to specific pollutant indicators rather than uniformly distributed across the dataset, allowing targeted treatment of these attributes in later analyses.

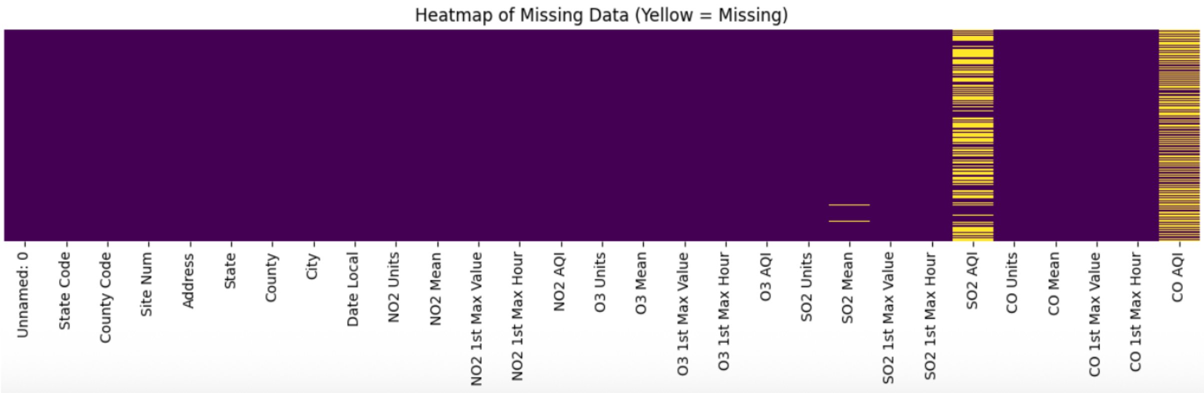

Figure A5. Heatmap of missing values across dataset attributes (yellow indicates missing cells)

A6. Correlation matrix of pollutant means, maximum values, and AQI

```
import matplotlib.pyplot as plt
import seaborn as sns

# Select only Column: pollutant mean, max, AQI
corr_cols = [c for c in df.columns if any(x in c for x in ["Mean","1st Max Value","AQI"])]

plt.figure(figsize=(10,8))
sns.heatmap(df[corr_cols].corr(numeric_only=True), annot=True, fmt=".2f", cmap="coolwarm", cbar=True)
plt.title("Correlation Matrix of Pollutant Means, Max Values, and AQI")
plt.tight_layout()
plt.show()
```

The correlation matrix illustrates the relationships among pollutant means, first maximum values, and AQI measures. As expected, extremely strong positive correlations are observed between each pollutant's first maximum value and its AQI (e.g., NO_2 1st Max Value – NO_2 AQI = 1.00; CO 1st Max Value – CO AQI = 1.00; SO_2 1st Max Value – SO_2 AQI = 0.99). This confirms that AQI values are almost directly derived from maximum concentration measures. Similarly, strong correlations exist between NO_2 Mean and NO_2 AQI (0.91) and CO Mean and CO AQI (0.67). Other key patterns include:

- **NO_2 and CO** show moderate to strong correlations (0.56–0.67), consistent with their common emission sources such as traffic and fuel combustion.
- **O_3** forms a tight internal cluster (O_3 Mean – O_3 AQI = 0.77; O_3 1st Max Value – O_3 AQI = 0.93) but exhibits negative correlations with NO_2 and CO (–0.29 to –0.43), reflecting its secondary formation through photochemical reactions rather than direct emission.
- **SO_2 attributes** are highly inter-correlated (0.83–0.99), but display only weak associations with other pollutants, suggesting SO_2 largely arises from distinct industrial sources.

Overall, the matrix in Figure A6 highlights both expected dependencies (AQI tightly linked with maxima) and pollutant-specific behaviours, offering useful insight for subsequent FP-Growth association rule mining.

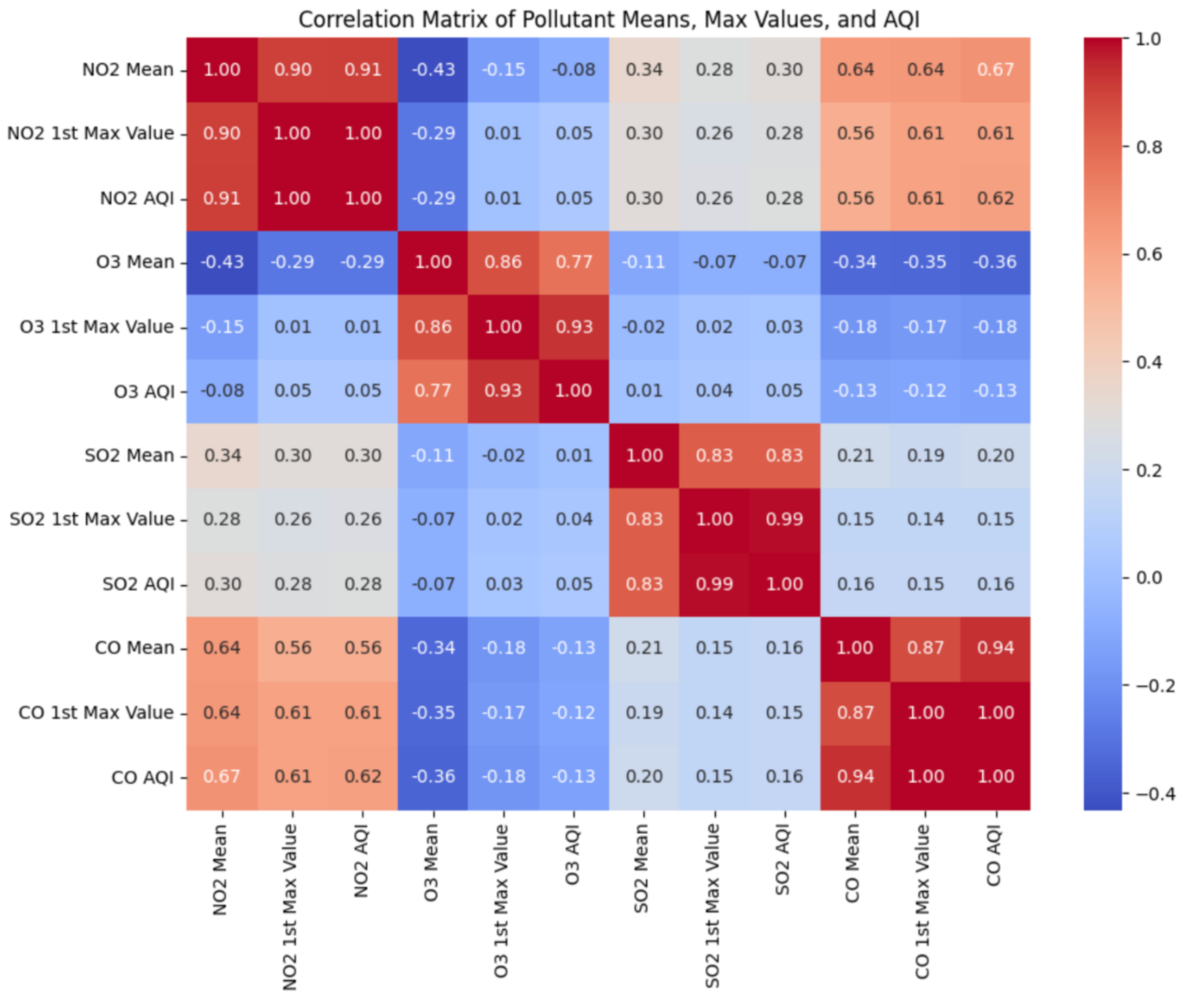

Figure A6. Correlation matrix of pollutant means, first maximum values, and AQI measures

A7. Histograms of pollutant mean concentrations

```
import matplotlib.pyplot as plt

# Filter for pollutant mean columns
mean_cols = ["NO2 Mean","SO2 Mean","CO Mean","O3 Mean"]

df[mean_cols].hist(figsize=(12,8), bins=30)
plt.suptitle("Histograms of Mean Concentrations of Pollutants (NO2, SO2, CO, O3)")
plt.tight_layout(rect=[0,0,1,0.95])
plt.show()
```

The histograms display the distributions of mean concentrations for NO_2, SO_2, CO, and O_3. Distinct patterns emerge across pollutants:

- **NO_2 Mean:** Strongly right-skewed, with most values between 0–30 ppb. A small number of extreme cases reach beyond 100 ppb, representing high-traffic or urban hotspots.
- **SO_2 Mean:** Almost all values are concentrated near zero, with the distribution dominated by very low concentrations (<5 ppb). Rare extreme outliers exceed 300 ppb, highlighting isolated industrial events.
- **CO Mean:** Highly skewed towards lower values, with the majority clustered around 0–1 ppm. Peaks above 7 ppm indicate severe but infrequent pollution episodes.
- **O_3 Mean:** The distribution approximates a bell curve, with most values between 0.01–0.06 ppm. This contrasts with the other pollutants, reflecting O_3's secondary formation through photochemical processes rather than direct emissions.

Together, these histograms confirm that NO_2, SO_2, and CO distributions are heavily skewed, while O_3 follows a more symmetric pattern. This variability is critical for downstream pattern mining, as categorical binning will be needed to balance these uneven distributions.

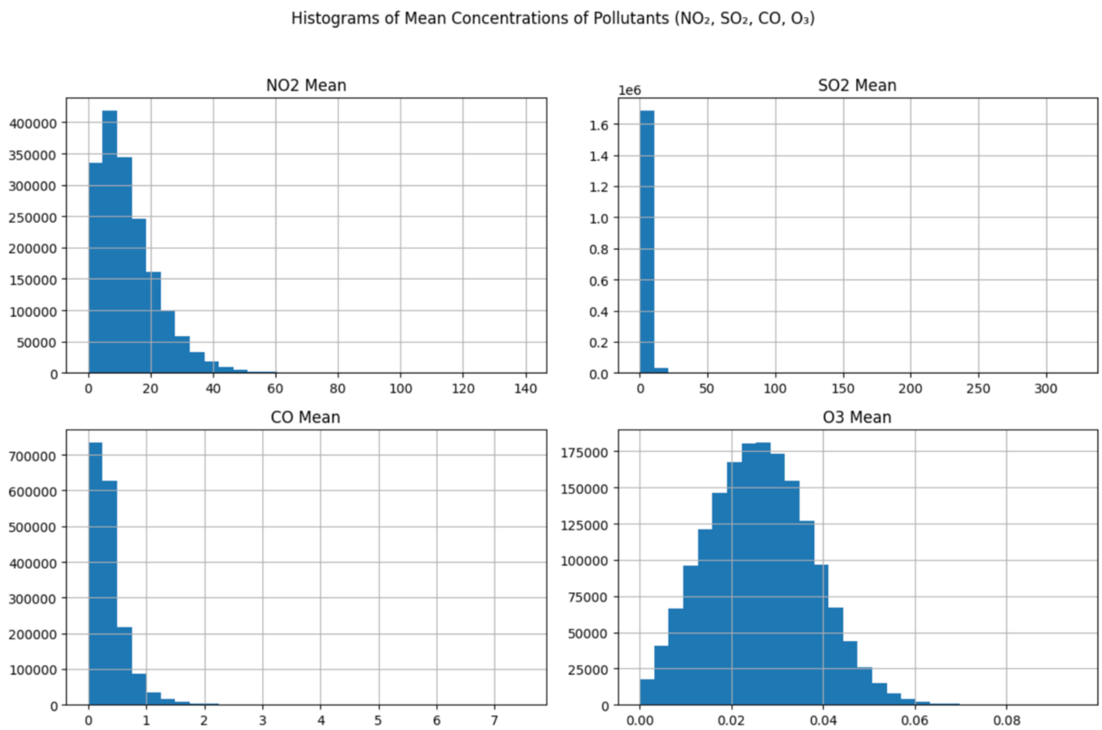

Figure A7. Histograms of mean concentrations for NO_2, SO_2, CO, and O_3

A8. Boxplots of AQI across the top 10 most polluted states

```
import matplotlib.pyplot as plt
import seaborn as sns
# Filtering for AQI columns
aqi_cols = ["NO2 AQI","O3 AQI","SO2 AQI","CO AQI"]
# Compute Overall AQI = max pollutant AQI
df["Overall AQI"] = df[aqi_cols].max(axis=1)
```

```
# avg AQI for each state and show Top-10
top10_states = (
    df.groupby("State")["Overall AQI"]
      .mean()
      .sort_values(ascending=False)
      .head(10)
      .index)
# prepare melt data for seaborn boxplot
plot_df = (
    df[df["State"].isin(top10_states)][["State"]+aqi_cols]
      .melt(id_vars="State", var_name="Pollutant AQI", value_name="AQI Score"))
# draw boxplots
plt.figure(figsize=(14,6))
sns.boxplot(data=plot_df, x="State", y="AQI Score", hue="Pollutant AQI")
plt.title("Boxplots of AQI Scores for Top 10 Most Polluted States")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

The boxplots illustrate AQI distributions for NO_2, O_3, SO_2, and CO across the ten most polluted states. Several insights emerge:

- **High variability**: States such as Arizona, North Carolina, and Nevada exhibit wide interquartile ranges and numerous outliers, reflecting frequent day-to-day fluctuations in pollutant levels.
- **Pollutant drivers**:
    - **O_3 AQI** consistently shows the highest medians and widest spreads, indicating its dominant role in driving overall AQI in most states.
    - **NO_2 AQI** also contributes strongly, with elevated medians across nearly all top 10 states, reflecting traffic and combustion-related emissions.
    - **SO_2 AQI** generally shows lower central values but with noticeable spread in states like Kentucky and Missouri, consistent with industrial and power-generation sources.
    - **CO AQI** remains the least variable, with low medians across states, though occasional peaks are evident.
- **Outliers**: High AQI outliers are especially visible for O_3 and NO_2, showing that extreme pollution episodes are recurrent across these states.

Overall, the figure highlights that O_3 and NO_2 are the most influential pollutants driving high AQI in the top 10 polluted states, while SO_2 and CO play more localized roles.

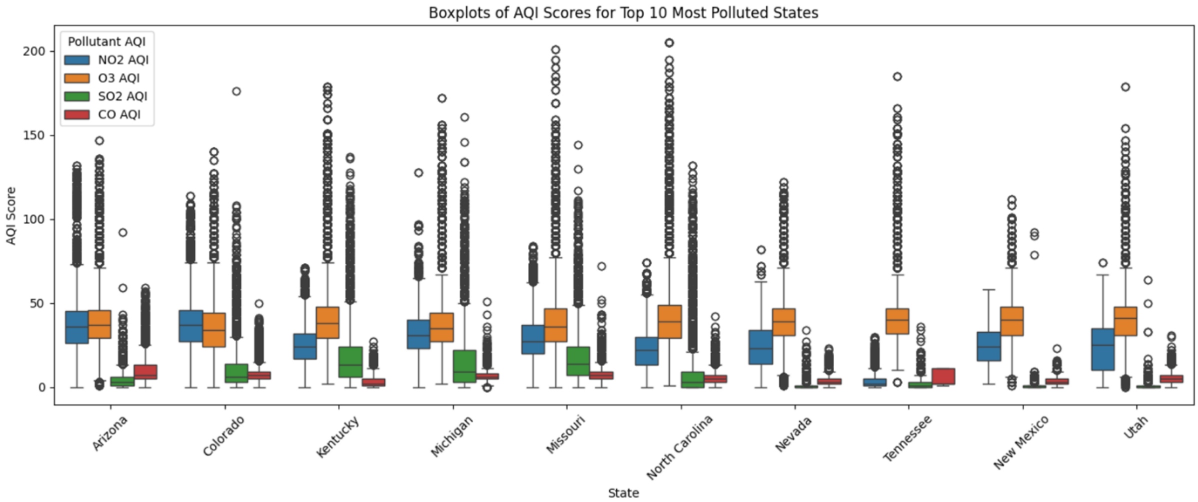

Figure A8. Boxplots of AQI values for NO_2, O_3, SO_2, and CO across the ten most polluted states

A9. Time series of median annual AQI trends (2000–2016)

```
import matplotlib.pyplot as plt

# modify Date
df["Date Local"] = pd.to_datetime(df["Date Local"], errors="coerce")
df["Year"] = df["Date Local"].dt.year
df = df[df["Year"].between(2000, 2016)]
# Filtering for AQI columns
aqi_cols = ["NO2 AQI","O3 AQI","SO2 AQI","CO AQI"]
# Compute yearly median
yearly_median = df.groupby("Year")[aqi_cols].median()
plt.figure(figsize=(12,6))
for col in aqi_cols:
    plt.plot(yearly_median.index, yearly_median[col], label=col)
plt.title("Median Annual AQI Trends by Pollutant (2000–2016)")
plt.xlabel("Year")
plt.ylabel("Median AQI")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

The time series of median annual AQI trends from 2000 to 2016 highlights contrasting behaviours across pollutants:

- **NO_2 AQI:** Shows a consistent decline over the study period, dropping from around 30 in 2000 to below 20 by 2016. This reduction reflects improved vehicle emission standards and urban air quality policies.
- **SO_2 AQI:** Displays the steepest downward trend, falling sharply after 2005 and approaching near-zero values after 2010. This aligns with industrial emission controls and the phase-out of high-sulphur fuels.
- **CO AQI:** Declines steadily between 2000 and 2010 before levelling off at low levels, indicating sustained improvements in combustion efficiency and regulatory enforcement.
- **O_3 AQI:** Unlike other pollutants, O_3 does not exhibit a clear decreasing trend. Its values remain relatively stable, fluctuating around 30–35, underscoring the challenge of managing photochemical smog.

Overall, the figure demonstrates substantial improvements for primary pollutants (NO_2, SO_2, CO), while O_3 remains persistent, highlighting its complex secondary formation processes.

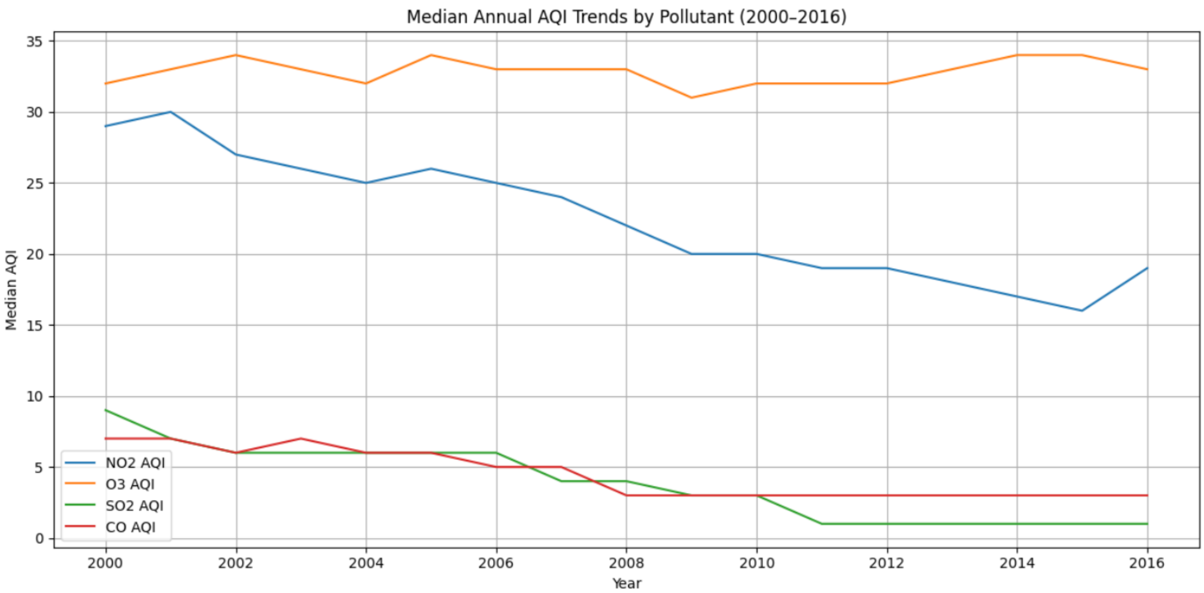

Figure A9. Median annual AQI trends by pollutant (2000–2016)

A10. Scatterplots of pollutant mean concentrations vs AQI

```
import matplotlib.pyplot as plt
import seaborn as sns
# tuple (mean, AQI) for each pollutant
pairs = [
    ("NO2 Mean", "NO2 AQI"),
    ("O3 Mean", "O3 AQI"),
    ("SO2 Mean", "SO2 AQI"),
    ("CO Mean", "CO AQI")]
fig, axes = plt.subplots(2, 2, figsize=(12,10))
axes = axes.flatten()
for i, (mean_col, aqi_col) in enumerate(pairs):
    sns.scatterplot(
        data=df, x=mean_col, y=aqi_col,
        alpha=0.3, s=10, ax=axes[i])
    axes[i].set_title(f"{mean_col} vs {aqi_col}")
    axes[i].set_xlabel(mean_col)
    axes[i].set_ylabel(aqi_col)
plt.suptitle("Scatterplots of Pollutant Mean Concentrations vs AQI", fontsize=16)
plt.tight_layout(rect=[0,0,1,0.97])
plt.show()
```

The scatterplots illustrate the relationships between mean pollutant concentrations and their corresponding AQI values:

- **NO_2 Mean vs NO_2 AQI:** A strong, near-linear relationship is visible, confirming that NO_2 concentration directly drives AQI values. Higher concentrations consistently translate to elevated AQI.
- **O_3 Mean vs O_3 AQI:** Also demonstrates a strong positive trend, but with greater dispersion. This reflects the secondary formation of ozone and its dependence on sunlight and precursor interactions.
- **SO_2 Mean vs SO_2 AQI:** Most points are concentrated at low concentration and low AQI values, with a few extreme outliers exceeding 200 AQI. This indicates that SO_2 usually plays a minor role in overall air quality, except during industrial pollution events.
- **CO Mean vs CO AQI:** Shows a monotonic increasing relationship, with most daily means clustering below 1 ppm. Rare but extreme spikes lead to disproportionately high AQI values.

Figure A10 scatterplots confirm that pollutant mean concentrations are strong predictors of AQI values, while also highlighting pollutant-specific behaviours. Ozone displays broader variability, whereas NO_2 and CO have tighter concentration-to-AQI mappings.

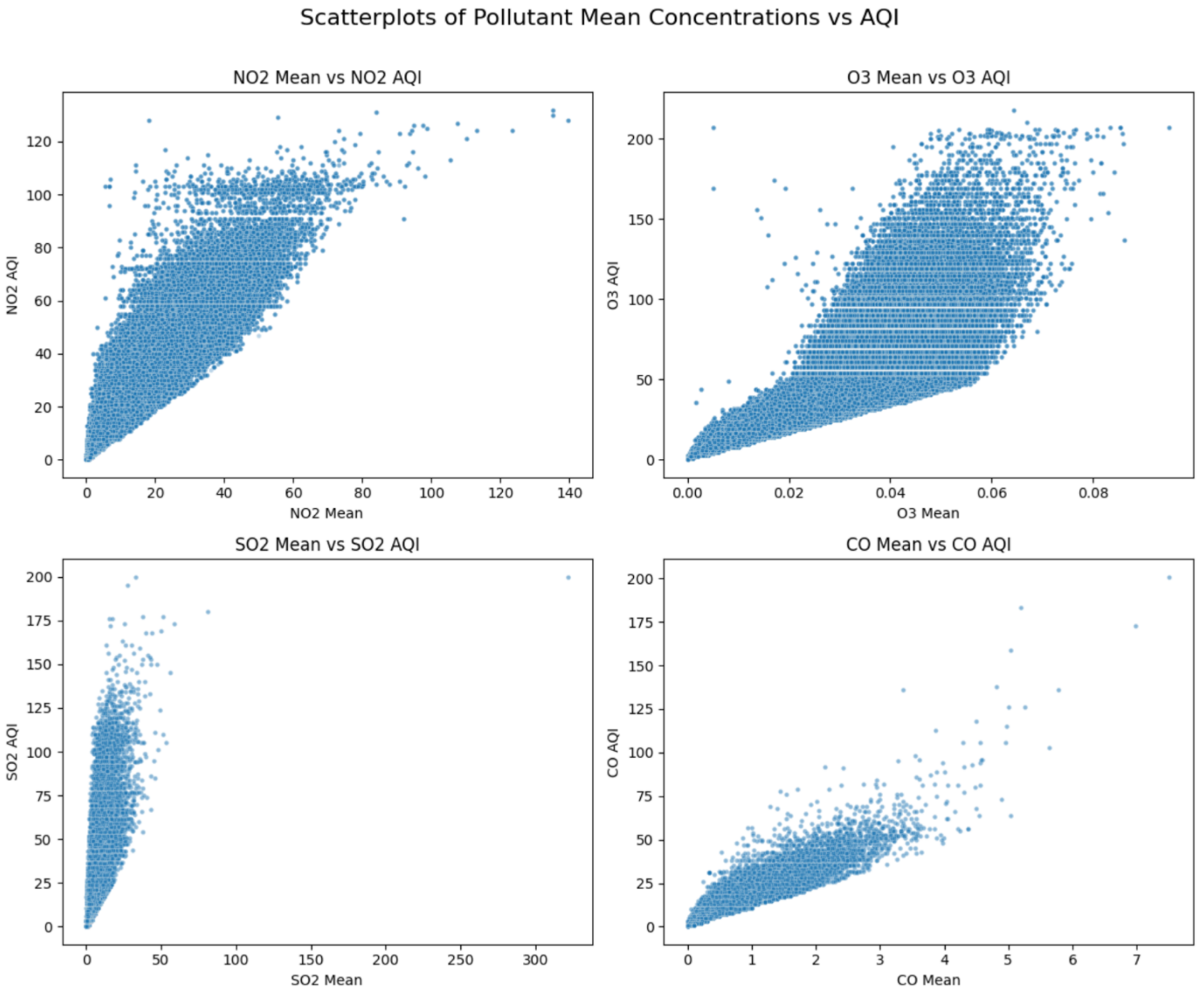

Figure A10. Scatterplots of mean pollutant concentrations versus AQI for NO_2, O_3, SO_2, and CO

Appendix B – Outlier Pattern Check

B1. Cleaned Dataset Check

```
# Setup & Load (outlier.ipynb)
import pandas as pd
df = pd.read_csv("pollution_cleaned.csv")
print("Shape of dataset:", df.shape)
print("\nColumns:", df.columns.tolist())
df.head()
```

The cleaned dataset contained 1,711,459 records and 29 attributes after filtering to U.S. states and removing Mexico and Colombia. Negative pollutant values were converted to NaN, and substantial missingness remains primarily in CO AQI and SO_2 AQI; rows will be dropped in subsequent steps as needed. Despite these adjustments, the dataset remains sufficiently large to support robust clustering and outlier detection.

```
# Exploratory check
import matplotlib.pyplot as plt
import seaborn as sns
pollutants = ["NO2 Mean","SO2 Mean","CO Mean","O3 Mean"]
plt.figure(figsize=(12,8))
for i, col in enumerate(pollutants, 1):
    plt.subplot(2,2,i)
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
plt.tight_layout()
plt.show()
```

An exploratory examination of the dataset revealed that pollutant distributions are highly skewed, particularly for NO_2, SO_2, and CO, with long tails extending to extreme values, while O_3 exhibited a more symmetric distribution. Correlation analysis also confirmed strong relationships between pollutant means and their respective AQI values. Boxplots (Figure B1) highlighted the presence of extreme outliers far beyond the interquartile ranges, reinforcing the need for anomaly detection methods to better identify and isolate such events.

B2. Data Distribution Check

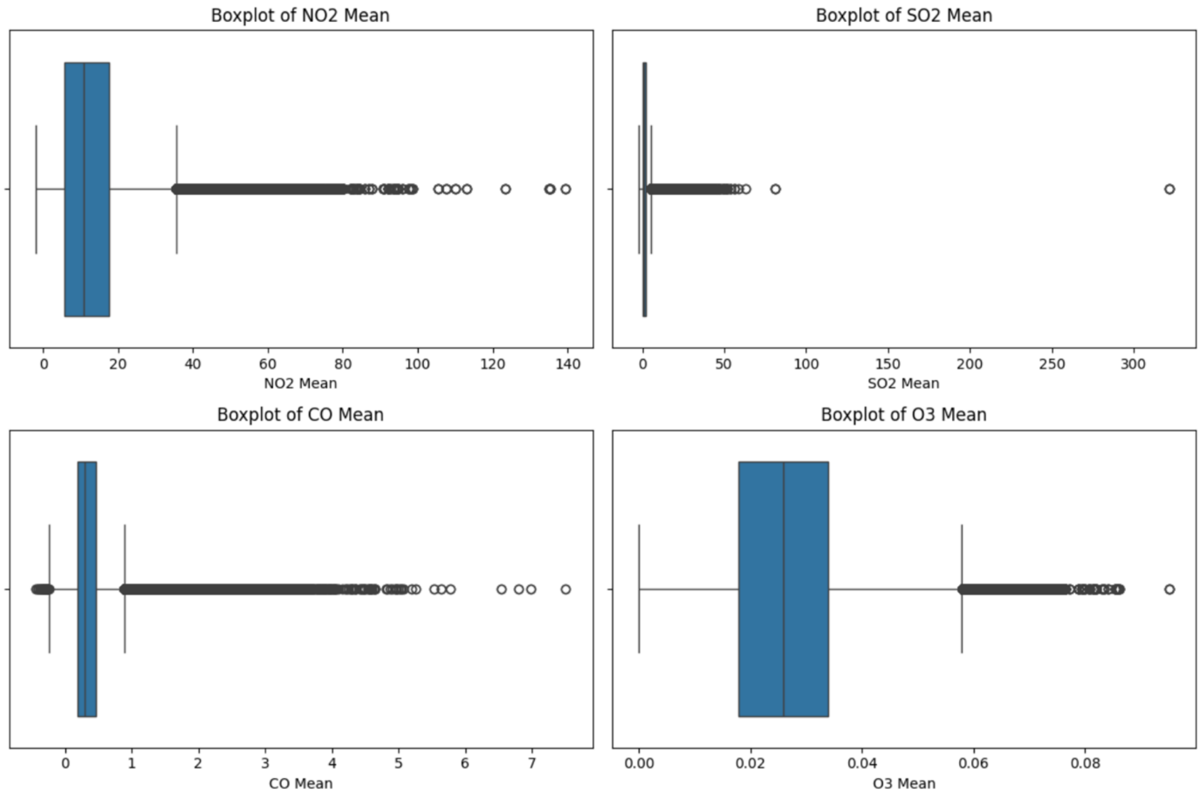

Figure B1. Boxplots of Pollutant Mean Concentrations (NO_2, SO_2, CO, O_3)

The boxplots illustrate the distribution of daily mean concentrations for NO_2, SO_2, CO, and O_3. Each pollutant exhibits a narrow interquartile range with numerous extreme values plotted as outliers. NO_2, SO_2, and CO display highly skewed distributions with long right tails, indicating occasional extreme pollution episodes, whereas O_3 presents a more symmetric spread but still includes notable high-end outliers. These plots confirm the presence of heavy-tailed distributions and justify the use of outlier detection methods in subsequent analysis.

To detect anomalies, the Cluster-Based Local Outlier Factor (CBLOF) algorithm was applied to annual subsets of the dataset. This approach ensures that long-term changes in pollution levels over time do not obscure anomalies specific to individual years. Igobwa et al. (2022) adopted a relatively high anomaly rate of around 10% when analysing hydro-climatic extremes. To provide a benchmark, we experimented with contamination rates of 10%, 5%, and 1%. At higher thresholds of 10% and 5%, the algorithm flagged too many points close to the central cluster, effectively overestimating anomalies. By contrast, a contamination rate of 5% yielded the most credible results, with outliers concentrated in the extreme tails of the distributions, where true abnormal pollution events are expected to occur.

For visualization, scatterplots were tuned for clarity. A marker size of s = 3 and transparency alpha = 0.3 were selected after cross-checking with boxplots to ensure that red outlier points remained visible without exaggerating their prevalence relative to the dense core clusters.

B3. Pollutants' Outlier Pattern Check

```python
from pyod.models.cblof import CBLOF
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np
# --- create column Year
df["Date Local"] = pd.to_datetime(df["Date Local"], errors="coerce")
df["Year"] = df["Date Local"].dt.year
# --- features for CBLOF
features = ["NO2 Mean", "NO2 AQI"]
years = sorted(df["Year"].dropna().unique())
n_cols = 4
n_rows = (len(years) + n_cols - 1) // n_cols
plt.figure(figsize=(20, n_rows*4))
for i, year in enumerate(years, 1):
    df_year = df.loc[df["Year"] == year, features].dropna()
    plt.subplot(n_rows, n_cols, i)
    if df_year.empty:
        plt.title(f"CBLOF Outliers on NO2 ({year}) - no data")
        plt.axis("off")
        continue
    # scale
    X = StandardScaler().fit_transform(df_year)
    # CBLOF
    model = CBLOF(n_clusters=5, contamination=0.05, random_state=42)
    model.fit(X)
    labels = model.predict(X)   # 0 = inlier, 1 = outlier
    # plot: inliers gray, outliers red
    inlier = labels == 0
    outlier = labels == 1
    plt.scatter(df_year["NO2 Mean"][inlier],  df_year["NO2 AQI"][inlier],
                c="#bbbbbb", s=6, alpha=0.4, label="inlier")
    plt.scatter(df_year["NO2 Mean"][outlier], df_year["NO2 AQI"][outlier],
                c="#E94D3C", s=8, alpha=0.7, label="outlier")
    plt.title(f"CBLOF Outliers on NO2 ({year})")
    plt.xlabel("NO2 Mean"); plt.ylabel("NO2 AQI")
plt.tight_layout()
plt.show()
```

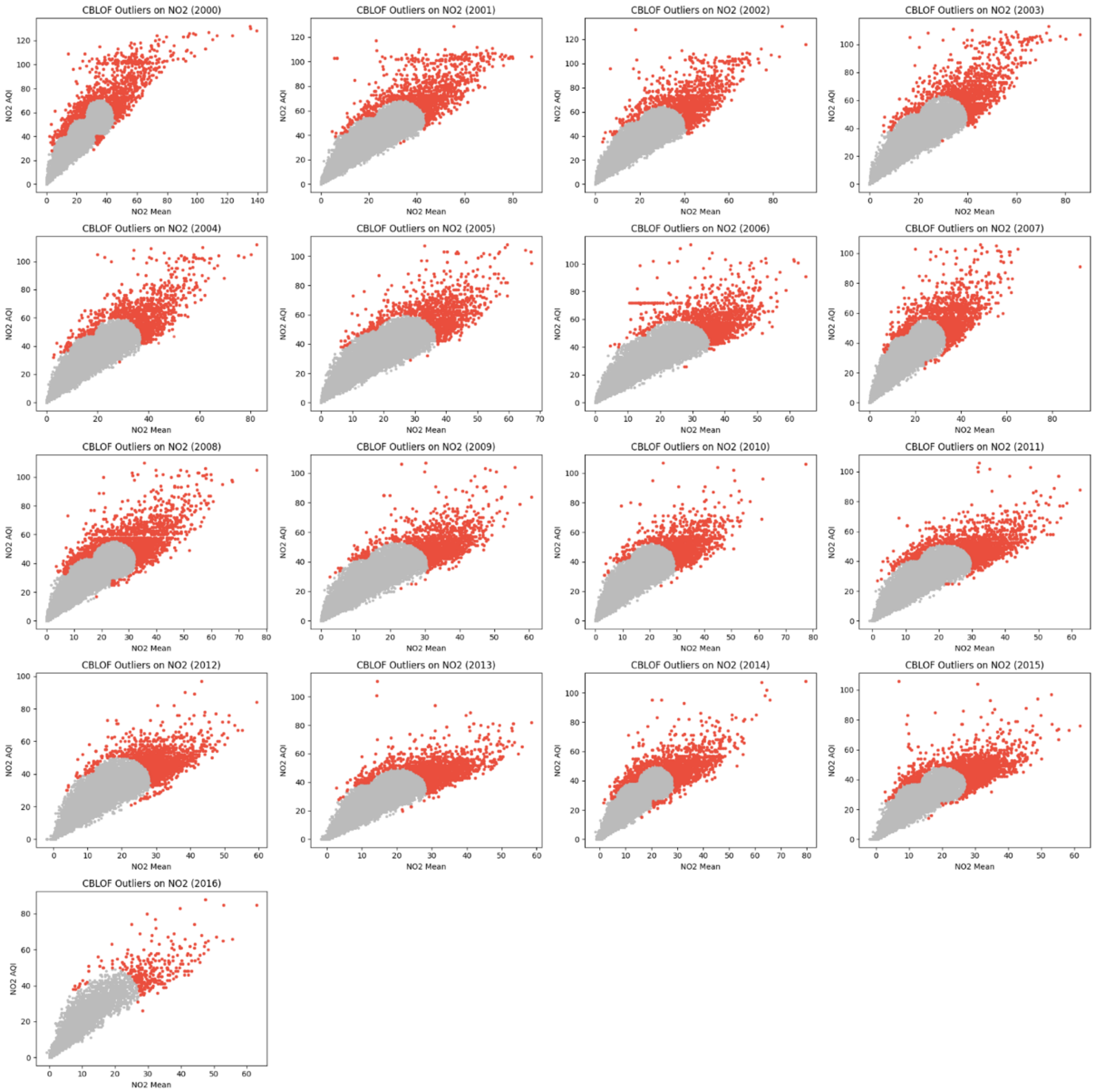

Figure B2. Annual CBLOF Outlier Detection on NO_2 (2000–2016)

The figure presents yearly CBLOF scatterplots of NO_2 Mean versus NO_2 AQI across 2000–2016. Gray points indicate inliers forming a dense core distribution, while red points denote outliers concentrated in the upper tail. The trend shows that while the main cluster gradually contracts over time, consistent with declining NO_2 levels, the relative positioning of outliers remains stable, representing extreme pollution episodes. This pattern highlights both the long-term reduction in average NO_2 concentrations and the persistence of occasional high-AQI anomalies.

```
from pyod.models.cblof import CBLOF
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np
# --- create column Year
df["Date Local"] = pd.to_datetime(df["Date Local"], errors="coerce")
df["Year"] = df["Date Local"].dt.year
# --- features for CBLOF
features = ["O3 Mean", "O3 AQI"]
years = sorted(df["Year"].dropna().unique())
n_cols = 4
n_rows = (len(years) + n_cols - 1) // n_cols
plt.figure(figsize=(20, n_rows*4))
for i, year in enumerate(years, 1):
    df_year = df.loc[df["Year"] == year, features].dropna()
    plt.subplot(n_rows, n_cols, i)
    if df_year.empty:
        plt.title(f"CBLOF Outliers on O3 ({year}) - no data")
        plt.axis("off")
        continue
    # scale
    X = StandardScaler().fit_transform(df_year)
    # CBLOF
    model = CBLOF(n_clusters=5, contamination=0.05, random_state=42)
    model.fit(X)
    labels = model.predict(X)   # 0 = inlier, 1 = outlier
    # plot: inliers gray, outliers red
    inlier = labels == 0
    outlier = labels == 1
    plt.scatter(df_year["O3 Mean"][inlier],  df_year["O3 AQI"][inlier],
                c="#bbbbbb", s=6, alpha=0.4, label="inlier")
    plt.scatter(df_year["O3 Mean"][outlier], df_year["O3 AQI"][outlier],
                c="#E94D3C", s=8, alpha=0.7, label="outlier")
    plt.title(f"CBLOF Outliers on O3 ({year})")
    plt.xlabel("O3 Mean"); plt.ylabel("O3 AQI")
plt.tight_layout()
plt.show()
```

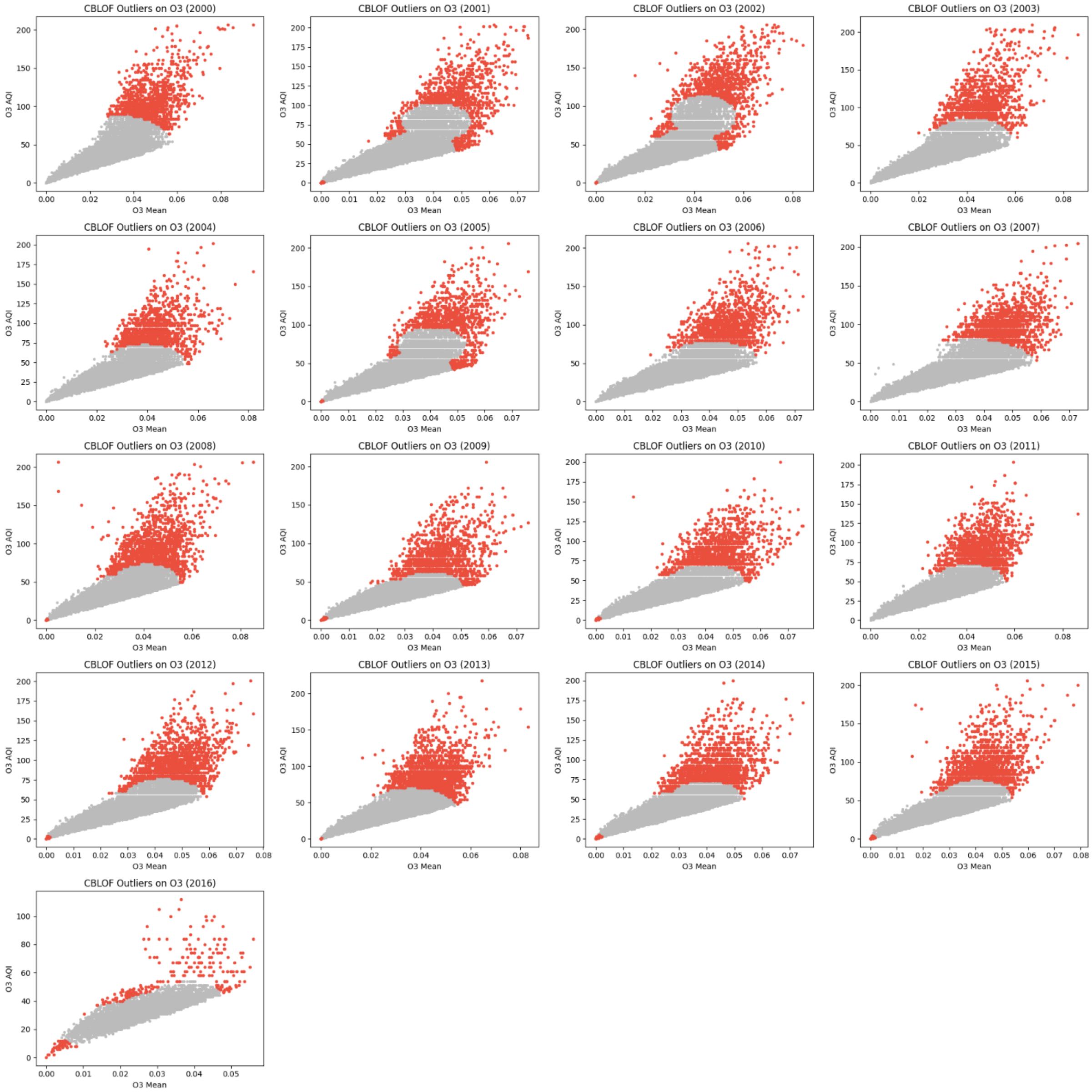

Figure B3. Annual CBLOF Outlier Detection for O_3 (2000–2016)

This figure presents yearly scatterplots of ozone (O_3) mean concentrations against their AQI values, with CBLOF highlighting outliers (red) relative to normal observations (gray). Outliers consistently appear at the upper tail of the distribution, capturing episodes of elevated O_3 that depart from the main concentration cluster. The results confirm that O_3 anomalies are widespread across years, with clustering structure stable but high-intensity outliers recurring, particularly in mid-range AQI bands. This highlights O_3's persistence as a secondary pollutant strongly influenced by photochemical activity.

```
from pyod.models.cblof import CBLOF
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np
# --- create column Year
df["Date Local"] = pd.to_datetime(df["Date Local"], errors="coerce")
df["Year"] = df["Date Local"].dt.year
# --- features for CBLOF
features = ["SO2 Mean", "SO2 AQI"]
years = sorted(df["Year"].dropna().unique())
n_cols = 4
n_rows = (len(years) + n_cols - 1) // n_cols
plt.figure(figsize=(20, n_rows*4))
for i, year in enumerate(years, 1):
    df_year = df.loc[df["Year"] == year, features].dropna()
    plt.subplot(n_rows, n_cols, i)
    if df_year.empty:
        plt.title(f"CBLOF Outliers on SO2 ({year}) - no data")
        plt.axis("off")
        continue
    # scale
    X = StandardScaler().fit_transform(df_year)
    # CBLOF
    model = CBLOF(n_clusters=5, contamination=0.05, random_state=42)
    model.fit(X)
    labels = model.predict(X)   # 0 = inlier, 1 = outlier
    # plot: inliers gray, outliers red
    inlier = labels == 0
    outlier = labels == 1
    plt.scatter(df_year["SO2 Mean"][inlier],  df_year["SO2 AQI"][inlier],
                c="#bbbbbb", s=6, alpha=0.4, label="inlier")
    plt.scatter(df_year["SO2 Mean"][outlier], df_year["SO2 AQI"][outlier],
                c="#E94D3C", s=8, alpha=0.7, label="outlier")
    plt.title(f"CBLOF Outliers on SO2 ({year})")
    plt.xlabel("SO2 Mean"); plt.ylabel("SO2 AQI")
plt.tight_layout()
plt.show()
```

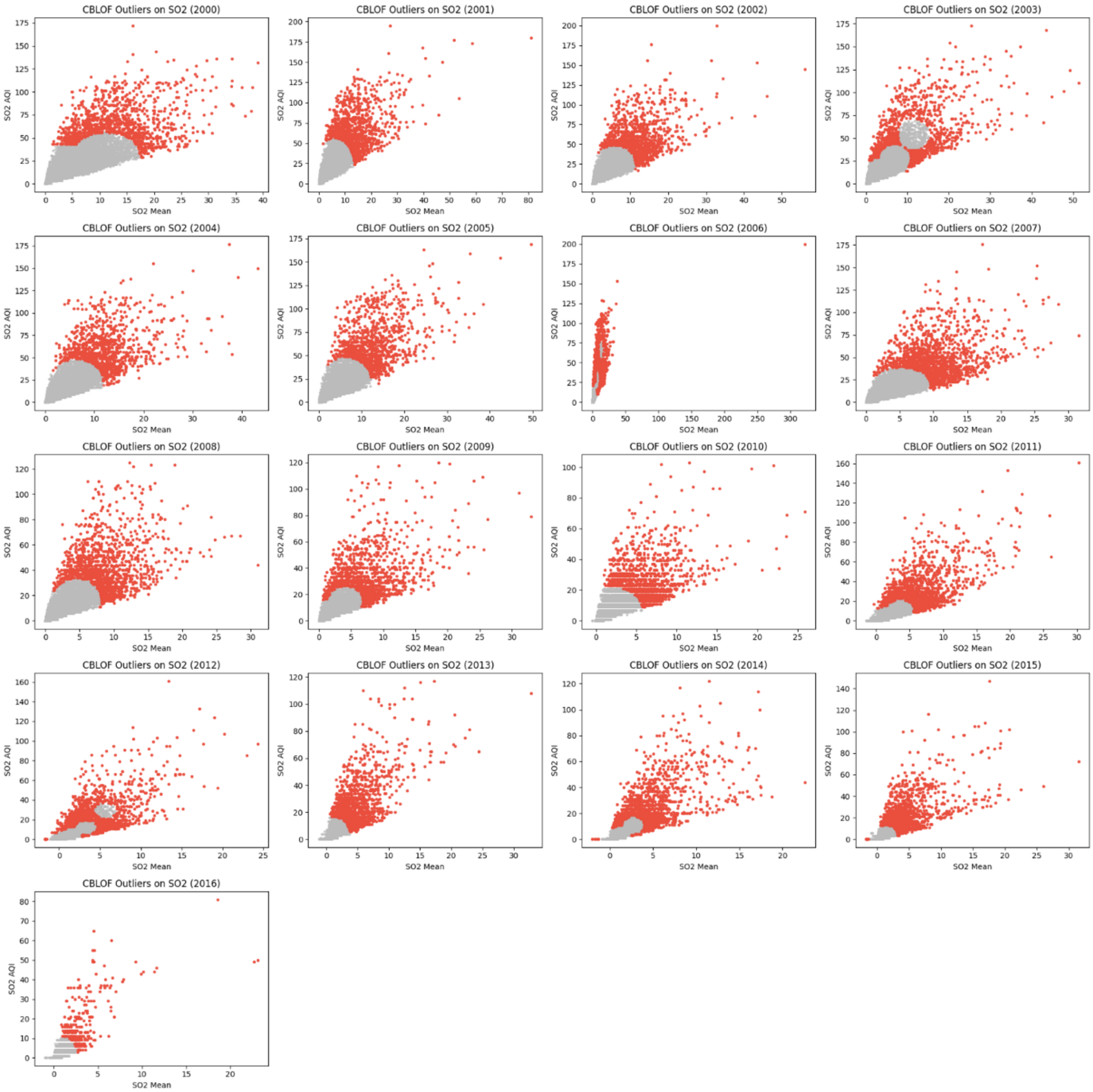

Figure B4. CBLOF Outlier Detection on SO_2 (2000–2016)

Annual CBLOF detection applied to SO_2 mean concentrations versus SO_2 AQI reveals a highly skewed distribution with recurring extreme anomalies. Outliers (red points) are most prevalent in the early 2000s, reflecting elevated SO_2 pollution events, while later years show fewer extreme deviations. This pattern is consistent with the overall downward trend of SO_2 emissions, suggesting that regulatory measures contributed to reducing the frequency of extreme SO_2 episodes.

```
from pyod.models.cblof import CBLOF
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import numpy as np
# --- create column Year
df["Date Local"] = pd.to_datetime(df["Date Local"], errors="coerce")
df["Year"] = df["Date Local"].dt.year
# --- features for CBLOF
features = ["CO Mean", "CO AQI"]
years = sorted(df["Year"].dropna().unique())
n_cols = 4
n_rows = (len(years) + n_cols - 1) // n_cols
plt.figure(figsize=(20, n_rows*4))
for i, year in enumerate(years, 1):
    df_year = df.loc[df["Year"] == year, features].dropna()
    plt.subplot(n_rows, n_cols, i)
    if df_year.empty:
        plt.title(f"CBLOF Outliers on CO ({year}) - no data")
        plt.axis("off")
        continue
    # scale
    X = StandardScaler().fit_transform(df_year)
    # CBLOF
    model = CBLOF(n_clusters=5, contamination=0.05, random_state=42)
    model.fit(X)
    labels = model.predict(X)   # 0 = inlier, 1 = outlier
    # plot: inliers gray, outliers red
    inlier = labels == 0
    outlier = labels == 1
    plt.scatter(df_year["CO Mean"][inlier],  df_year["CO AQI"][inlier],
            c="#bbbbbb", s=6, alpha=0.4, label="inlier")
    plt.scatter(df_year["CO Mean"][outlier], df_year["CO AQI"][outlier],
            c="#E94D3C", s=8, alpha=0.7, label="outlier")
    plt.title(f"CBLOF Outliers on CO ({year})")
    plt.xlabel("CO Mean"); plt.ylabel("CO AQI")
plt.tight_layout()
plt.show()
```

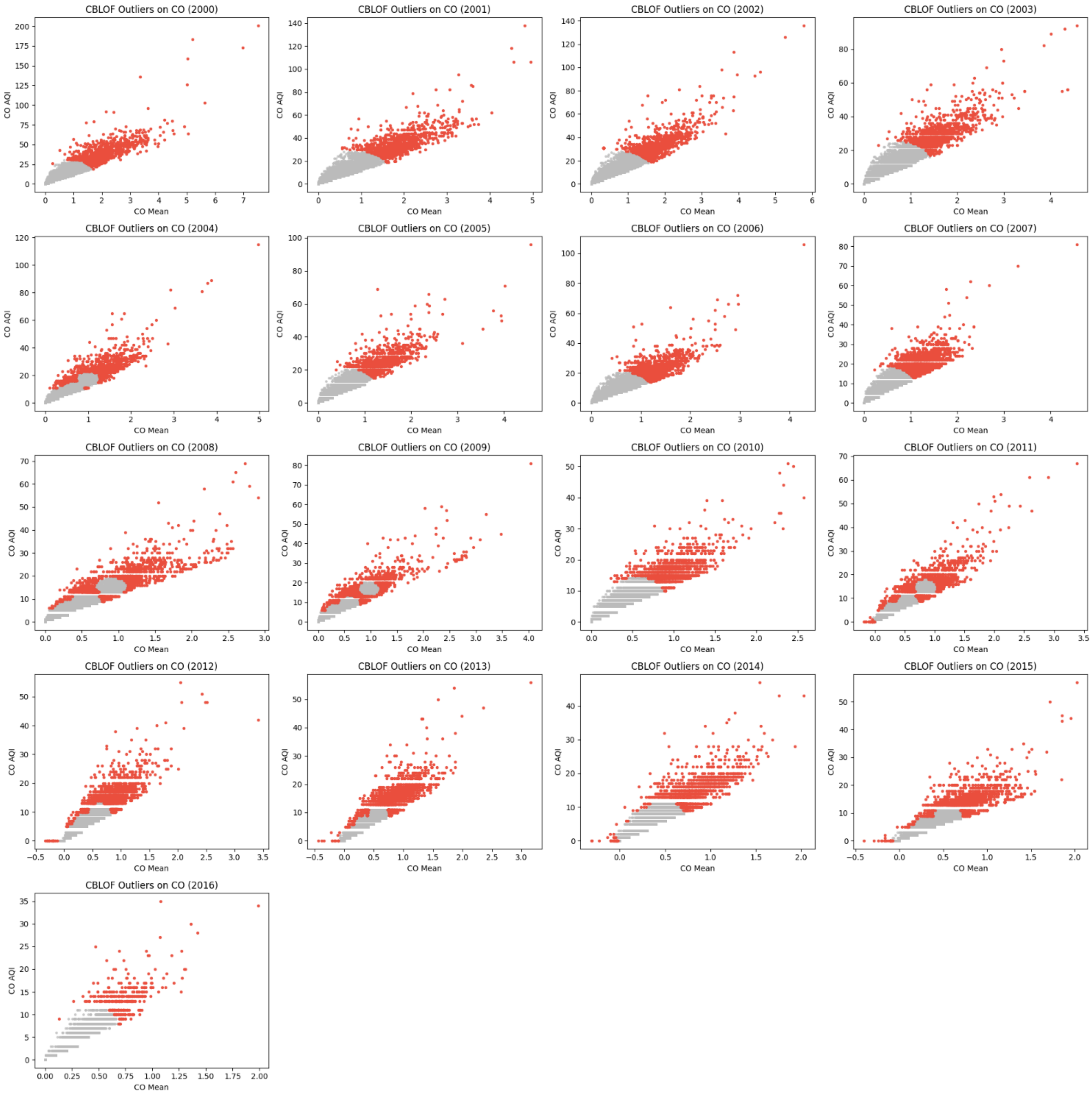

Figure B5. CBLOF Outliers on CO (2000–2016)

Scatterplots of CO mean concentration versus CO AQI with annual CBLOF outlier detection. Gray points represent inliers, and red points indicate detected outliers. Outliers are consistently concentrated in the upper tail of CO distributions, especially in the early 2000s when extreme values were more frequent. Over time, both mean levels and extreme anomalies decreased, reflecting gradual improvements in air quality.

Summary

Overall, the annual outlier analysis confirms that pollutant distributions are stable enough for higher-level grouping tasks. While certain years (e.g., 2006 SO_2 and CO) show lower baseline values, their outlier structures remain consistent, ensuring reliability for dimensionality reduction and clustering. Building on this, the next stage applies PCA and K-Means to uncover city-level pollution patterns and seasonal groupings.

Appendix C – PCA and Clustering

C1. Data Preparation

The cleaned dataset was used as the input for PCA and clustering. Initially, daily records were aggregated by city and year to construct representative pollutant profiles. However, because the number of cities is very large, the results would be difficult to interpret and less actionable for policymaking. In addition, environmental regulations in the United States are often implemented at the **state** level rather than by individual cities. For these reasons, the analysis was adjusted to aggregate pollutant profiles by state, ensuring that the clustering results remain both interpretable and directly relevant to future policy considerations.

Prior to PCA, features were standardized using z-score normalization to ensure comparability across pollutants with differing units and ranges.

```
# C1. Setup & Load
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from pathlib import Path
print(df.shape)
df.head()
# C2. select columns
id_cols = ["State", "County", "City", "Year"]
pollutant_mean_cols = ["NO2 Mean","SO2 Mean","CO Mean","O3 Mean"]
aqi_cols = ["NO2 AQI","SO2 AQI","CO AQI","O3 AQI","Overall AQI"]
use_cols = id_cols + pollutant_mean_cols + aqi_cols
df_use = df[use_cols].copy()
# cut NaN rows out
df_use = df_use.dropna(subset=pollutant_mean_cols + aqi_cols)
print("Rows after final dropna:", df_use.shape[0])
# C3. create key each state
df_use["StateKey"] = df_use["State"].str.strip()
AGG_FN = "median"   # or "mean"
agg_map = {c: AGG_FN for c in pollutant_mean_cols + aqi_cols}
# group state-year -> years profile for each state
g_state_year = (
    df_use
    .groupby(["StateKey","Year"], as_index=False)
    .agg(agg_map))
print("State-Year rows:", g_state_year.shape)
g_state_year.head(10)
```

For PCA and clustering, the cleaned dataset was loaded and reduced to relevant columns. A state identifier key was then created. After removing rows with missing AQI values, **428,062** valid records remained, which were aggregated to yield **473** state-year profiles for subsequent analysis.

```
# C4. save new state-year csv for PCA by year
state_year_csv = OUT_DIR / "state_year_profiles.csv"
g_state_year.to_csv(state_year_csv, index=False)
print("Saved:", state_year_csv)
# C5. collect long-term profile each state
g_state_long = (
    df_use
    .groupby(["StateKey"], as_index=False)
    .agg(agg_map))
print("State-long rows:", g_state_long.shape)
state_long_csv = OUT_DIR / "state_long_profiles.csv"
g_state_long.to_csv(state_long_csv, index=False)
print("Saved:", state_long_csv)
# C6. feature matrix + Scaling
SOURCE = "state_year"   # or "state_long"
if SOURCE == "state_year":
    df_src = g_state_year.copy()
    id_keep = ["StateKey","Year"]
else:
    df_src = g_state_long.copy()
    id_keep = ["StateKey"]
feature_cols = pollutant_mean_cols + aqi_cols
X_raw = df_src[feature_cols].values.astype(float)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)
print("X_raw shape:", X_raw.shape, "| X_scaled shape:", X_scaled.shape)
# C7. save feature matrix + Scaling for PCA/K-Means
np.save(OUT_DIR / f"{SOURCE}_X_scaled.npy", X_scaled)
df_src[id_keep].to_csv(OUT_DIR / f"{SOURCE}_ids.csv", index=False)
pd.Series(feature_cols).to_csv(OUT_DIR / "feature_cols.txt", index=False, header=False)
import joblib
joblib.dump(scaler, OUT_DIR / "standard_scaler.joblib")
print("Artifacts saved to:", OUT_DIR.resolve())
```

In **#C4–C5**, state-year profiles and long-term state profiles were aggregated and saved, yielding 473 state-year rows and 45 state-long rows. Finally, feature matrices were standardized and exported (**#C6–C7**), resulting in an 9-variable dataset with 473 observations. These outputs provide ready-to-use inputs for the subsequent PCA and K-Means analysis.

C2. PCA Analysis

```
import numpy as np
import pandas as pd
X = np.load(OUT_DIR / "state_year_X_scaled.npy")
ids = pd.read_csv(OUT_DIR / "state_year_ids.csv")
feat = pd.read_csv(OUT_DIR / "feature_cols.txt", header=None)[0].tolist()
print(X.shape, len(ids), feat)
print("zero-variance columns:", [f for f, v in zip(feat, X.var(axis=0)) if np.isclose(v, 0)])
```

Cleaned dataset was imported for PCA/Clustering analysis. The scaled pollutant features (X_scaled), identifiers (ids), and feature names (feat). A check for zero-variance columns confirmed none were present, ensuring that all variables contribute meaningful variation.

```
# C8. PCA fit & scree
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
X = np.load(OUT_DIR / "state_year_X_scaled.npy")
ids = pd.read_csv(OUT_DIR / "state_year_ids.csv")
feat = pd.read_csv(OUT_DIR / "feature_cols.txt", header=None)[0].tolist()
pca = PCA(n_components=X.shape[1], random_state=42)
PC = pca.fit_transform(X)
# Scree
plt.figure(figsize=(6,4))
plt.plot(np.arange(1, X.shape[1]+1), np.cumsum(pca.explained_variance_ratio_)*100, marker="o")
plt.xlabel("Number of Components"); plt.ylabel("Cumulative Explained Variance (%)")
plt.title("Scree (State-Year)")
plt.grid(True, ls="--", alpha=0.4)
plt.show()
# Loadings table for interpreting
loadings = pd.DataFrame(
    pca.components_.T,
    index=feat,
    columns=[f"PC{i+1}" for i in range(X.shape[1])])
loadings.head(10)
```

PCA was applied on the state-year matrix. The scree plot (Figure C1) shows that the first four principal components together explain more than 95% of the cumulative variance, confirming that dimensionality can be reduced to four without major information loss. Table C1 presents the loadings of pollutants on each component.

For example, **PC1** loads strongly on NO_2 Mean (0.42), NO_2 AQI (0.39), CO AQI (0.41), and CO AQI (0.40), capturing a combined signal of NO_2 and SO_2. **PC2** is dominated by O_3 AQI (0.62), O_3 Mean (0.52), and Overall AQI (0.56), highlighting O_3 as the primary driver of this dimension. **PC3** emphasizes SO_2 Mean (0.60) and SO_2 AQI (0.53), while **PC4** has high contributions from CO Mean (0.53).

Table C1. Loadings table for Cumulative Explained Variance (%)

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8	PC9
NO2 Mean	0.417210	-0.028603	-0.066480	-0.457316	0.315242	-0.061641	0.056641	-0.177804	0.688158
SO2 Mean	0.351842	0.021547	0.600156	0.248053	0.304786	0.454500	0.367858	0.103859	-0.091946
CO Mean	0.355825	0.027488	-0.421558	0.528553	0.130894	-0.063271	0.096649	-0.605863	-0.134185
O3 Mean	-0.241362	0.528513	0.089507	0.139861	0.682698	-0.269211	-0.300304	0.074745	-0.022936
NO2 AQI	0.406780	0.047167	-0.115051	-0.491511	0.123219	0.226893	-0.393835	-0.035740	-0.595348
SO2 AQI	0.375143	0.097344	0.526459	0.168375	-0.389101	-0.416876	-0.443191	-0.124631	0.084541
CO AQI	0.401037	0.043779	-0.387663	0.331091	-0.037660	0.053224	-0.173567	0.718561	0.163225
O3 AQI	-0.101493	0.624245	-0.065530	0.018615	-0.331257	0.589685	-0.179730	-0.189710	0.263864
Overall AQI	0.199856	0.561549	-0.068548	-0.224726	-0.216302	-0.370574	0.589912	0.125745	-0.203962

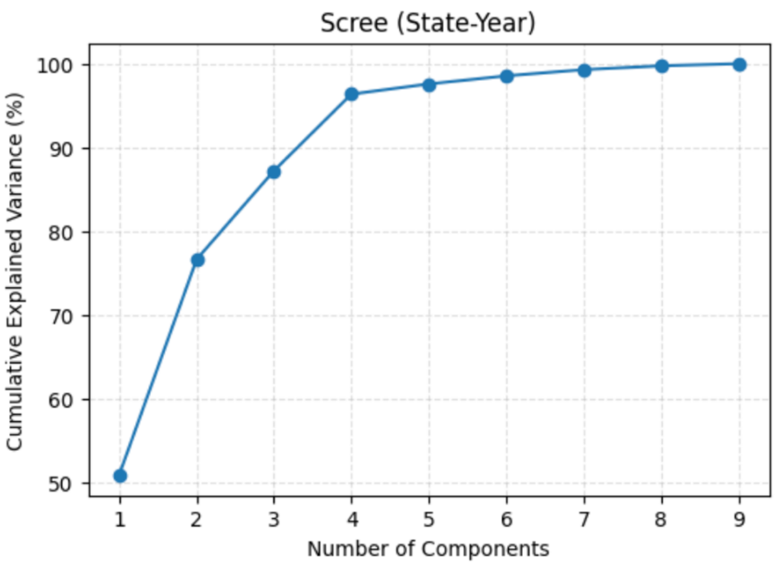

Figure C1. Scree plot for State-Year components

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# use first 4 PCs (covering >95% cumulative variance)
X_for_cluster = PC[:, :4]
scores = []
for k in range(2, 10):
    km = KMeans(n_clusters=k, n_init=20, random_state=42).fit(X_for_cluster)
    scores.append(silhouette_score(X_for_cluster, km.labels_))
pd.Series(scores, index=range(2,10))
```

Table C2. K-values and scores for K-Means using Silhouette scores

K-value	Score
2	**0.365405**
3	0.251933
4	0.250545
5	0.230664
6	0.251961
7	0.257975
8	0.260208
9	0.254093

K-Means clustering was then performed using the first 4 principal components shown in Table C2. Silhouette scores (GeeksforGeeks, 2025) across k=2 to k=10 suggest the most interpretable clustering structure is achieved at lower k values, with k=2 producing the highest scores, concluding that the data point fits well.

C3. K-Means

```
# === C9. K-Means on 4 PCs (state-year), k=2 -> 2 clusters
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# (1) Load scaled matrix + IDs (STATE-YEAR)
X = np.load(OUT_DIR / "state_year_X_scaled.npy")
ids = pd.read_csv(OUT_DIR / "state_year_ids.csv")   # has columns: StateKey, Year
# (2) PCA -> keep first 4 PCs (>95% CEV from scree)
pca = PCA(n_components=4, random_state=42)
PC = pca.fit_transform(X)  # shape: (n_rows, 4)
# (3) Fit K-Means (k=2)
km = KMeans(n_clusters=2, n_init=20, random_state=42).fit(PC)
labels = km.labels_
sil = silhouette_score(PC, labels)
print(f"Silhouette (k=2): {sil:.4f}")
print("Cluster sizes:\n", pd.Series(labels).value_counts().sort_index())
# (4) Attach labels & PCs back to StateKey+Year
df_clusters = ids.copy()
for i in range(4):
    df_clusters[f"PC{i+1}"] = PC[:, i]
df_clusters["Cluster_k2"] = labels
# Save csv
out_csv = OUT_DIR / "state_year_k2_labels.csv"
df_clusters.to_csv(out_csv, index=False)
print("Saved:", out_csv)
# (5) Quick viz: PC1–PC2 colored by cluster (+ centroids)
plt.figure(figsize=(7,6))
scatter = plt.scatter(df_clusters["PC1"], df_clusters["PC2"],
                      c=df_clusters["Cluster_k2"], cmap="coolwarm", s=18, alpha=0.7)
# Centroids in PC space >> to visualise
centroids = km.cluster_centers_
plt.scatter(centroids[:,0], centroids[:,1], marker="X", s=160, edgecolor="k", linewidth=1.0)
plt.title("State–Year clusters on PC1–PC2 (k=2)")
plt.xlabel("PC1"); plt.ylabel("PC2")
plt.grid(True, ls="--", alpha=0.3)
plt.legend(*scatter.legend_elements(), title="Cluster", loc="best")
plt.show()
```

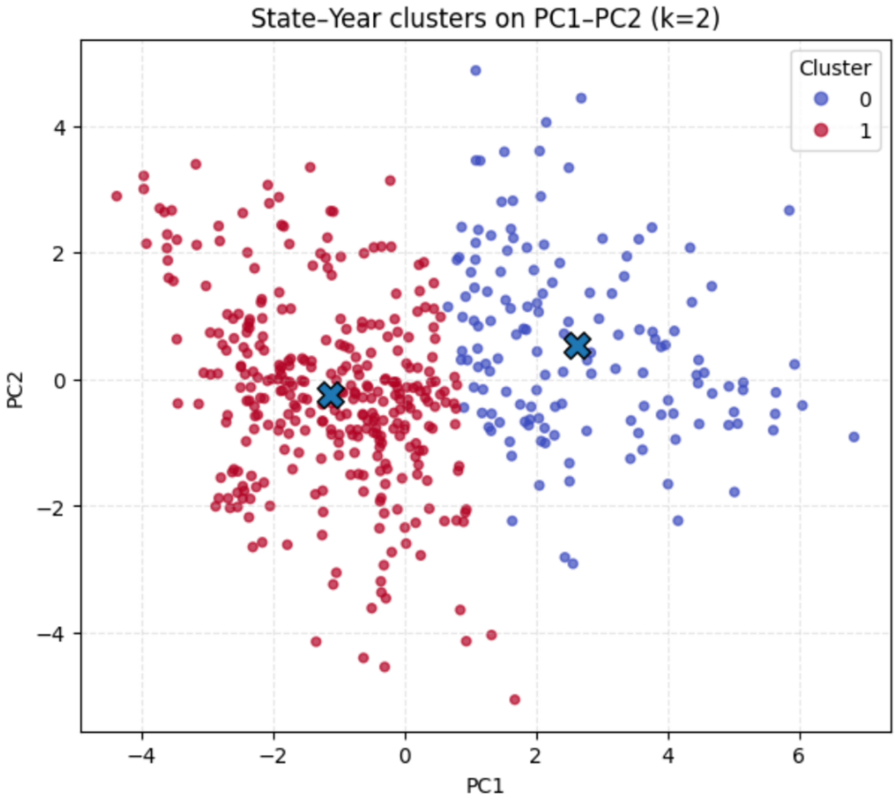

Figure C2. K-Means State-Year clusters (k=2)

Figure C2 visualises the K-Means clustering of U.S. states based on their pollutant profiles, projected onto the first 2 principal components. With k=2 (0.3654), the model separates the state-year observations into 2 distinct groups: Cluster 0 (blue, 143 dots) and Cluster 1 (red, 330 dots). The clusters are well balanced around their centroids, suggesting 2 interpretable pollution regimes. This representation provides a basis for linking states’ air quality characteristics with potential differences in policy or emission-control strategies.

C3. K-Means

```
# Check main cluster for each state (mode)
state_majority = (
    labels.groupby("StateKey")["Cluster_k2"]
          .agg(lambda s: s.value_counts().idxmax())
          .reset_index()
          .rename(columns={"Cluster_k2": "MajorCluster"})
)
print(state_majority)
```

Table C3. Major cluster of each State

No.	State Key	Major Cluster
0	Alabama	1
1	Alaska	1
2	Arizona	0
3	Arkansas	1
4	California	1
5	Colorado	0
6	Connecticut	1
7	Delaware	1
8	Florida	1
9	Georgia	1
10	Hawaii	1
11	Idaho	1
12	Illinois	1
13	Indiana	1
14	Iowa	1
15	Kansas	1
16	Kentucky	1
17	Louisiana	1
18	Maine	1
19	Maryland	1
20	Massachusetts	1
21	Michigan	0
22	Minnesota	1
23	Missouri	0
24	Nevada	1
25	New Hampshire	1
26	New Jersey	0
27	New Mexico	1
28	New York	0

No.	State Key	Major Cluster
29	North Carolina	1
30	North Dakota	1
31	Ohio	1
32	Oklahoma	1
33	Oregon	1
34	Pennsylvania	1
35	Rhode Island	1
36	South Carolina	1
37	South Dakota	1
38	Tennessee	1
39	Texas	1
40	Utah	1
41	Virginia	1
42	Washington	1
43	Wisconsin	1
44	Wyoming	1

Each state was assigned to a dominant cluster across years. Most of all states consistently fell into Cluster 1, including large states such as California, Texas, and Florida, highlighting a broad common pollution profile. A smaller group of states, including Arizona, Colorado, Michigan, Missouri, New Jersey, and New York, were classified into Cluster 0, indicating distinct air quality patterns. This division into two clusters underscores stable grouping patterns suitable for comparative policy analysis, as summarized in Table C3.

```
import pandas as pd
from pathlib import Path
# --- Setup ---
OUT_DIR = Path("./_pca_artifacts_v2")
OUT_DIR.mkdir(exist_ok=True)
# --- Load existing artifacts ---
labels   = pd.read_csv(OUT_DIR / "state_year_k2_labels.csv")     # StateKey, Year, PC1..PC4, Cluster_k2
profiles = pd.read_csv(OUT_DIR / "state_year_profiles.csv")      # StateKey, Year, NO2 Mean..Overall AQI
# --- Merge labels with profiles ---
dfm = labels.merge(
    profiles,
    on=["StateKey", "Year"],
    how="left",
    validate="many_to_one")
# --- Define pollutant columns of interest ---
poll_cols = [
    "NO2 Mean","SO2 Mean","CO Mean","O3 Mean",
    "NO2 AQI","SO2 AQI","CO AQI","O3 AQI","Overall AQI"]
# --- Compute mean pollutant profile per state ---
state_means = (
    dfm.groupby("StateKey")[poll_cols]
      .mean()
      .reset_index()
      .sort_values("Overall AQI", ascending=False))
state_means.to_csv(OUT_DIR / "state_means.csv", index=False)
print("Saved:", OUT_DIR / "state_means.csv")
# --- Compute mean pollutant profile per cluster ---
cluster_profiles = (
    dfm.groupby("Cluster_k2")[poll_cols]
      .mean()
      .round(2))
cluster_profiles.to_csv(OUT_DIR / "cluster_profiles.csv")
print("Saved:", OUT_DIR / "cluster_profiles.csv")
print("\nTop 10 states by Overall AQI:")
print(state_means)
print("\nCluster profiles:")
print(cluster_profiles)
```

Table C4. Pollutants' concentrations in each cluster

Cluster	NO_2 Mean	SO_2 Mean	CO Mean	O_3 Mean	NO_2 AQI	SO_2 AQI	CO AQI	O_3 AQI	Overall AQI
0	16.02	2.70	0.43	0.02	30.71	10.50	6.86	33.37	40.85
1	8.27	0.75	0.23	0.03	17.47	2.19	3.29	32.94	35.00

From Table C4, cluster 0 represents states with relatively **higher pollution**, showing modest levels of NO_2, SO_2, and CO and an average overall AQI of about 41, indicating cleaner air conditions. In contrast, Cluster 1 captures states with consistently **lower pollutant** concentrations, particularly NO_2 and SO_2, and an elevated overall AQI of around 35, reflecting poorer air quality more typical of industrialized or urban regions.

To further interpret the clustering results, state-level mean pollutant concentrations and AQI values were computed by averaging across all available years. This summary table provides a clearer view of each state's long-term air quality profile, allowing for direct comparison with the overall pollutant characteristics of each cluster. Such aggregation helps bridge the gap between annual fluctuations and broader structural patterns relevant for policy evaluation.

Summary

The PCA and clustering analysis of U.S. state-year pollution profiles. After cleaning and aggregation, 473 state-year profiles were standardized for PCA. The first four components explained over 95% of variance, with PC1 reflecting NO_2 and CO, and PC2 dominated by O_3. K-Means clustering identified 2 stable groups: Cluster 0 with higher pollutants (Overall AQI approximately 41) and Cluster 1 with lower levels (Overall AQI approximately 35). Most states consistently belonged to one cluster, with high-pollution states like Arizona and Colorado standing out. These results provide a clear basis for linking pollution regimes with state-level policy contexts.

Table C5. Pollutants' concentrations in each state descending ranked by Overall AQI

	StateKey	NO2 Mean	SO2 Mean	CO Mean	O3 Mean	NO2 AQI	SO2 AQI	CO AQI	O3 AQI	Overall AQI
5	Colorado	18.730741	1.287905	0.412378	0.023131	38.058824	9.294118	7.058824	33.088235	44.147059
2	Arizona	17.964660	1.100347	0.401927	0.024534	35.882353	3.235294	7.411765	37.235294	43.705882
40	Utah	14.285417	0.435417	0.318056	0.029205	26.833333	0.500000	5.166667	36.833333	42.500000
29	North Carolina	9.900703	1.259695	0.349020	0.030113	22.205882	5.264706	5.294118	40.117647	41.676471
27	New Mexico	10.565196	0.576667	0.182778	0.032610	24.400000	0.600000	2.600000	40.200000	41.200000
44	Wyoming	2.580719	0.184262	0.057682	0.039217	8.000000	0.000000	1.125000	40.437500	40.562500
24	Nevada	10.660035	0.215436	0.163958	0.032235	24.650000	0.150000	2.600000	37.600000	40.500000
13	Indiana	11.709498	2.512420	0.337623	0.029574	24.323529	9.176471	4.941176	38.852941	40.411765
23	Missouri	13.259341	2.585733	0.400160	0.027245	26.923077	12.692308	5.846154	35.692308	39.500000
26	New Jersey	17.464482	2.771998	0.380804	0.022464	31.785714	9.928571	5.500000	28.857143	39.357143
38	Tennessee	1.454024	0.636866	0.418452	0.036369	3.000000	1.142857	4.571429	38.785714	38.785714
34	Pennsylvania	10.492997	2.978620	0.205719	0.026461	21.411765	10.176471	3.294118	35.294118	38.235294
21	Michigan	15.534420	1.946181	0.304948	0.025504	30.000000	10.125000	5.375000	32.750000	37.937500
32	Oklahoma	6.102685	0.459621	0.117525	0.030676	13.235294	1.470588	1.647059	37.235294	37.823529
28	New York	17.513245	3.226728	0.338399	0.022863	30.029412	9.882353	5.176471	28.617647	37.794118
41	Virginia	8.950679	1.998837	0.310648	0.028443	18.041667	5.750000	4.166667	35.500000	37.625000
16	Kentucky	10.711480	2.119643	0.152381	0.026205	23.071429	9.000000	2.714286	34.714286	37.500000
39	Texas	11.022039	0.464142	0.238971	0.024946	23.382353	2.000000	3.764706	32.823529	37.294118
12	Illinois	14.242909	1.958545	0.377696	0.022529	26.823529	8.588235	5.882353	29.647059	37.117647
7	Delaware	10.206582	0.782891	0.231944	0.027243	20.666667	2.000000	3.000000	33.833333	37.000000
6	Connecticut	8.340420	0.664701	0.259688	0.028505	18.350000	1.950000	3.300000	33.700000	36.300000
4	California	10.726972	0.816700	0.356128	0.025989	21.529412	2.294118	5.705882	32.470588	36.294118
19	Maryland	9.292854	1.617534	0.274053	0.027067	20.454545	5.545455	4.272727	33.500000	36.272727
36	South Carolina	2.666667	0.462245	0.143014	0.031105	6.285714	2.142857	1.857143	36.000000	36.142857
11	Idaho	7.717150	0.390519	0.179861	0.025153	20.000000	0.666667	3.000000	33.666667	35.666667
15	Kansas	9.908948	2.124758	0.335417	0.024906	21.029412	5.823529	5.058824	30.794118	35.470588
17	Louisiana	12.794283	1.528235	0.383002	0.022387	24.588235	5.705882	5.411765	30.411765	35.382353
35	Rhode Island	5.676647	0.261648	0.202778	0.030208	11.666667	0.500000	2.333333	35.000000	35.333333
31	Ohio	9.659081	1.685765	0.260559	0.026211	18.875000	8.000000	3.250000	31.375000	34.875000
22	Minnesota	5.686979	0.577718	0.219531	0.027932	14.500000	0.500000	2.500000	33.500000	34.750000
8	Florida	6.692889	0.198898	0.427906	0.025694	14.882353	1.647059	5.705882	33.705882	34.529412
9	Georgia	11.171040	0.414773	0.283036	0.019596	24.857143	0.857143	5.142857	30.285714	34.357143
0	Alabama	9.283542	0.624943	0.185590	0.022089	22.250000	3.250000	3.000000	31.750000	34.000000
25	New Hampshire	6.583899	1.017157	0.338542	0.027881	13.291667	3.750000	4.833333	32.916667	33.958333
3	Arkansas	8.641812	1.376271	0.394792	0.025310	20.350000	2.500000	5.800000	32.300000	33.800000
37	South Dakota	4.570492	0.429679	0.195000	0.029925	9.200000	0.200000	2.000000	33.400000	33.400000
14	Iowa	6.471529	0.245286	0.212500	0.027288	13.045455	0.090909	2.545455	32.363636	32.818182
20	Massachusetts	17.569467	2.001861	0.286971	0.020093	28.656250	5.687500	4.218750	24.625000	32.687500
42	Washington	10.651563	0.526489	0.216667	0.021363	22.250000	1.000000	3.000000	29.000000	31.250000
30	North Dakota	3.837817	0.158730	0.185880	0.025751	9.722222	0.000000	2.000000	29.333333	30.000000
43	Wisconsin	15.815217	2.020834	0.304167	0.018937	26.000000	3.500000	4.000000	23.500000	29.250000
18	Maine	3.631866	0.527941	0.204545	0.024987	8.363636	1.181818	2.272727	27.727273	28.863636
10	Hawaii	2.812525	0.863352	0.251250	0.025129	7.600000	1.000000	3.200000	28.000000	28.000000
33	Oregon	8.883586	0.925513	0.253009	0.019991	16.944444	1.444444	4.000000	25.388889	26.833333
1	Alaska	8.333333	4.093025	0.327083	0.011375	17.500000	10.500000	5.000000	17.500000	25.000000

Appendix D – States Clustering Visualisation

D1. Set States' Overall AQI on Map

```
# 1) set states
import pandas as pd
import plotly.express as px
# Load state means
df = pd.read_csv("./_pca_artifacts_v2/state_means.csv")
# State abbreviations
abbr = {
 'Alabama':'AL','Alaska':'AK','Arizona':'AZ','Arkansas':'AR','California':'CA','Colorado':'CO',
 'Connecticut':'CT','Delaware':'DE','Florida':'FL','Georgia':'GA','Hawaii':'HI','Idaho':'ID',
 'Illinois':'IL','Indiana':'IN','Iowa':'IA','Kansas':'KS','Kentucky':'KY','Louisiana':'LA',
 'Maine':'ME','Maryland':'MD','Massachusetts':'MA','Michigan':'MI','Minnesota':'MN','Missouri':'MO',
 'Mississippi':'MS','Montana':'MT','Nebraska':'NE','Nevada':'NV','New Hampshire':'NH','New Jersey':'NJ',
 'New Mexico':'NM','New York':'NY','North Carolina':'NC','North Dakota':'ND','Ohio':'OH','Oklahoma':'OK',
 'Oregon':'OR','Pennsylvania':'PA','Rhode Island':'RI','South Carolina':'SC','South Dakota':'SD',
 'Tennessee':'TN','Texas':'TX','Utah':'UT','Vermont':'VT','Virginia':'VA','Washington':'WA',
 'West Virginia':'WV','Wisconsin':'WI','Wyoming':'WY'}
df["state"] = df["StateKey"].map(abbr)
# 2) choropleth (Overall AQI)
fig = px.choropleth(
    df.dropna(subset=["state"]),
    locations="state", locationmode="USA-states",
    color="Overall AQI",
    range_color=[20, 50],   # หรือใช้ dynamic: [df["Overall AQI"].min(), df["Overall AQI"].max()]
    color_continuous_scale="YlOrRd",
    scope="usa",
    labels={"Overall AQI": "Overall AQI"},
    title="Overall AQI by State (mean of 2000–2016)")
# layout tuning
fig.update_layout(
    width=1200, height=800,
    title=dict(font=dict(size=24)),
    coloraxis_colorbar=dict(
        title="Overall AQI", titlefont=dict(size=18),
        tickfont=dict(size=14)),
    geo=dict(bgcolor='rgba(0,0,0,0)'))
fig.show()
```

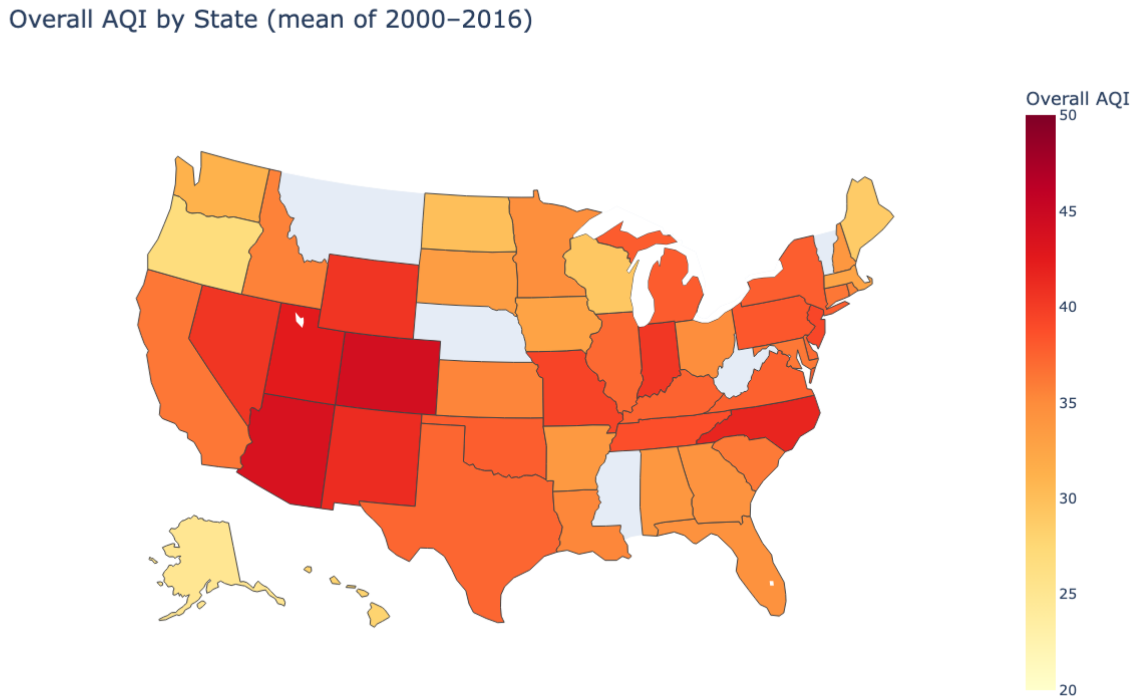

Figure D1. Overall AQI by States (2000-2016)

Figure D1 presents the choropleth of average Overall AQI across U.S. states from 2000–2016. The AQI scale was restricted to the range of 20–50, since state-level means lie from around 25 to 45; using the standard 0–100 scale would compress colours and obscure meaningful variation. The map shows that western and industrialized states such as Arizona, Colorado, and Missouri exhibit relatively higher AQI values, while many northern and coastal states remain in lighter shades, indicating cleaner air. This highlights a clear spatial divide where pollutant burdens cluster in central and urban-heavy regions.

Appendix E. WEKA analysis and Hard Code calculation

E1. Workflow overview

1. Data preparation for WEKA pipeline
   Copy state_year_profiles.csv from Python preparation to WEKA program, and remove identifiers (StateKey, Year), standardize all features (z-score). Using PrincipalComponents (variance covered = 0.95) and then AddCluster (Unsupervised) with SimpleKMeans (k = 2, distance = Euclidean, seed = 42).

2. Hard-code K-means (using NumPy)
   Re-implement K-means without Python libraries and then compute Sum of Squared Errors (SSE) within-cluster and Silhouette from scratch for validation.

E2. WEKA steps and outputs

1. PCA: After Standardize, the PrincipalComponents (variance Covered = 0.95) was applied and transformed attributes No. 1–4 in Figure D1 represent PC1–PC4 (the PCA scores).
2. K-means: Add a nominal cluster attribute via AddCluster and select SimpleKMeans with K = 2 and seed = 42, consistent with the Python setup. Figure D3 shows the WEKA Clusterer output including initial centroids, final centroids, and Clustered Instances. and then export state_year_k2_labels_WEKA.csv and kmeans_weka_output.txt to Jupyter Notebook for future comparison.

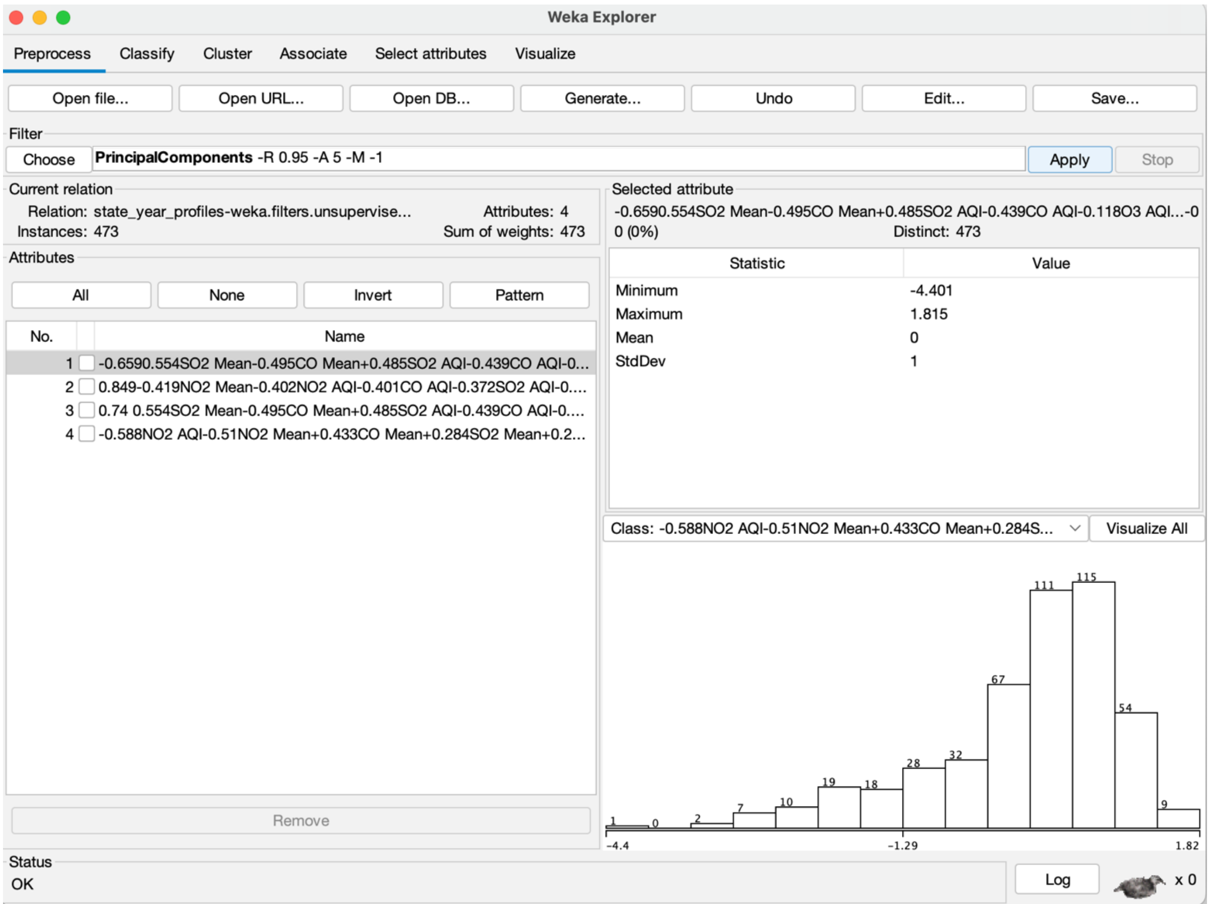

Figure D1. Standardize (Z-score) for all attributes on Unsupervised preprocessing and Principal Component

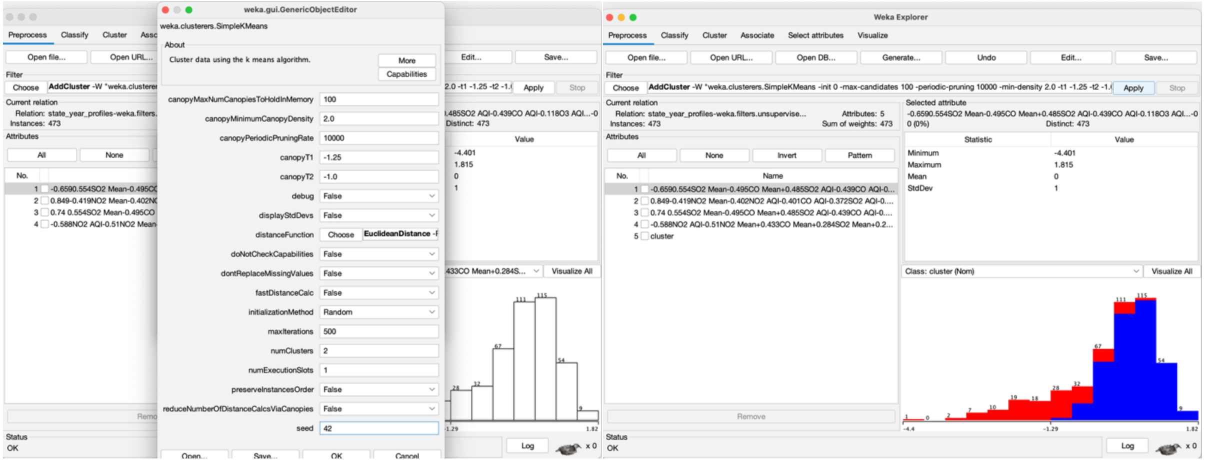

Figure D2. Simple K-Means result (K=2, seed = 42)

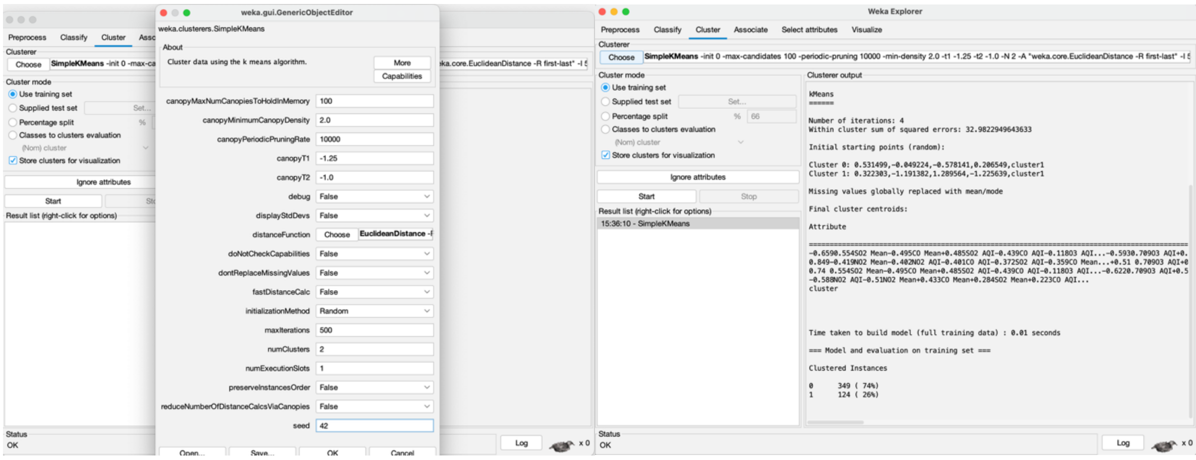

Figure D3. Using Cluster for Simple K-Means (K=2, seed = 42)

E3. WEKA labels and the Python (sklearn) labels verify shapes

```
from sklearn.metrics import silhouette_score
import numpy as np, pandas as pd
# Silhouette
sil_weka = silhouette_score(X, lab_weka, metric="euclidean")
sil_sk   = silhouette_score(X, lab_sk,   metric="euclidean")
# % overlap (original & switch 0<->1)
agree_raw  = (lab_weka == lab_sk).mean()
agree_flip = ((1 - lab_weka) == lab_sk).mean()
print("Silhouette (WEKA):   ", round(sil_weka, 3))
print("Silhouette (sklearn):", round(sil_sk,   3))
print("% same (WEKA orginal) :", round(agree_raw*100, 1), "%")
print("% same (WEKA flip):", round(agree_flip*100, 1), "%")
# if flip/switch version % is better then go for it applying as weka_aligned
```

```
weka_aligned = lab_weka if agree_raw >= agree_flip else (1 - lab_weka)
aligned_note = "as-is" if agree_raw >= agree_flip else "flipped to match sklearn"
print("WEKA labels used for comparison:", aligned_note)
```

The high agreement after flipping (approximately 89.6%) confirms that WEKA and sklearn discovered essentially the same two groups. While Silhouette values are close (WEKA = 0.331 vs sklearn = 0.353), indicating similar clustering across tools.

E4. Hard-coded K-means (without libraries)

```
import numpy as np, pandas as pd
X   = np.load("state_year_X_scaled.npy").astype(float)
ids = pd.read_csv("state_year_ids.csv")
# K-means Hard code
def kmeans_scratch(X, k=2, max_iter=500, seed=42):
    rng = np.random.default_rng(seed)
    N = X.shape[0]
    # select actual point as a beginning centroid
    C = X[rng.choice(N, size=k, replace=False)].copy()
    labels = np.zeros(N, dtype=int)
    for it in range(max_iter):
        # assign step, distance from every point to centroid (N x k)
        d = np.stack([np.linalg.norm(X - c, axis=1) for c in C], axis=1)
        new_labels = d.argmin(axis=1)
        if it > 0 and np.all(new_labels == labels):
            break
        labels = new_labels
        # update step
        for j in range(k):
            pts = X[labels == j]
            if len(pts) > 0:
                C[j] = pts.mean(axis=0)
            else:
                # if there's empty cluster, random again
                C[j] = X[rng.integers(N)]
    # Sum of Squared Errors -> SSE (within-cluster sum of squares)
    sse = sum(((X[labels==j] - C[j])**2).sum() for j in range(k))
    return labels, C, float(sse)
# Silhouette Hard code without sklearn
from math import inf
def silhouette_score_from_scratch(X, labels):
    X = np.asarray(X, float); labels = np.asarray(labels, int)
    N = len(labels)
    # pairwise euclidean distances
    aa = (X*X).sum(1)[:,None]; bb = aa.T; ab = X @ X.T
```

```
    D2 = aa + bb - 2*ab; D2[D2<0] = 0.0
    D  = np.sqrt(D2, dtype=float)
    s_vals = np.zeros(N, float)
    for i in range(N):
        li = labels[i]
        same = (labels==li)
        a = D[i, same].sum()/(same.sum()-1) if same.sum()>1 else 0.0
        b = min(D[i, labels==lj].mean() for lj in np.unique(labels) if lj!=li)
        s_vals[i] = (b - a) / max(a, b) if max(a,b)>0 else 0.0
    return float(np.nanmean(s_vals))
#run
lab_hard, cent_hard, sse_hard = kmeans_scratch(X, k=2, max_iter=500, seed=42)
sil_hard = silhouette_score_from_scratch(X, lab_hard)
#save
out = ids.copy()
out["k2_hard"] = lab_hard
out.to_csv("state_year_k2_labels_HARD.csv", index=False)
#compare with others
agree_hard_sk   = (lab_hard == lab_sk).mean()
agree_hard_weka = (lab_hard == weka_aligned).mean()
print("HARD-CODE SSE:", round(sse_hard,2))
print("HARD-CODE Silhouette:", round(sil_hard,3))
print("% similar to sklearn:", round(agree_hard_sk*100,1), "%")
print("% similar to WEKA   :", round(agree_hard_weka*100,1), "%")
```

The result shows

```
HARD-CODE SSE: 2787.68
HARD-CODE Silhouette: 0.353
% similar to sklearn: 99.4 %
% similar to WEKA   : 89.9 %
```

The hard-coded algorithm reproduces sklearn almost perfectly (99.4% similarly, Silhouette = 0.353), validating implementation ability. While the hard-coded vs WEKA (similarity approximately 89.9%) has some differences between platforms, but still acceptable. SSE within cluster from hard-coded (2787.68) quantifies within-cluster compactness.